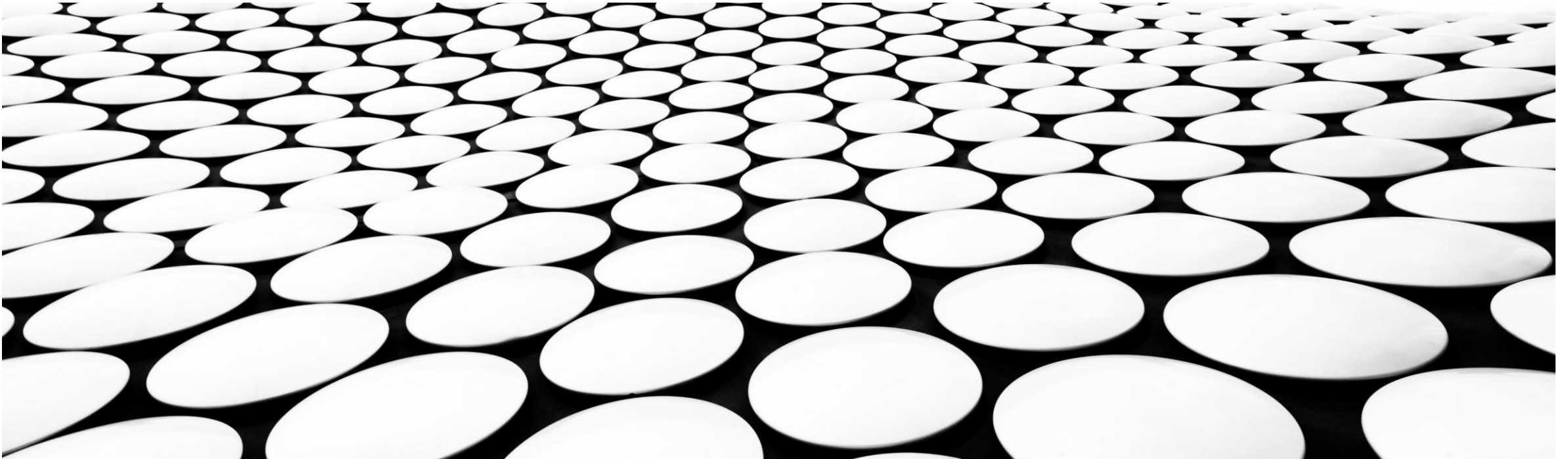
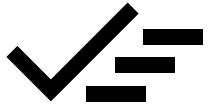


DATA TYPES AND STORAGE

Exploring different types of databases





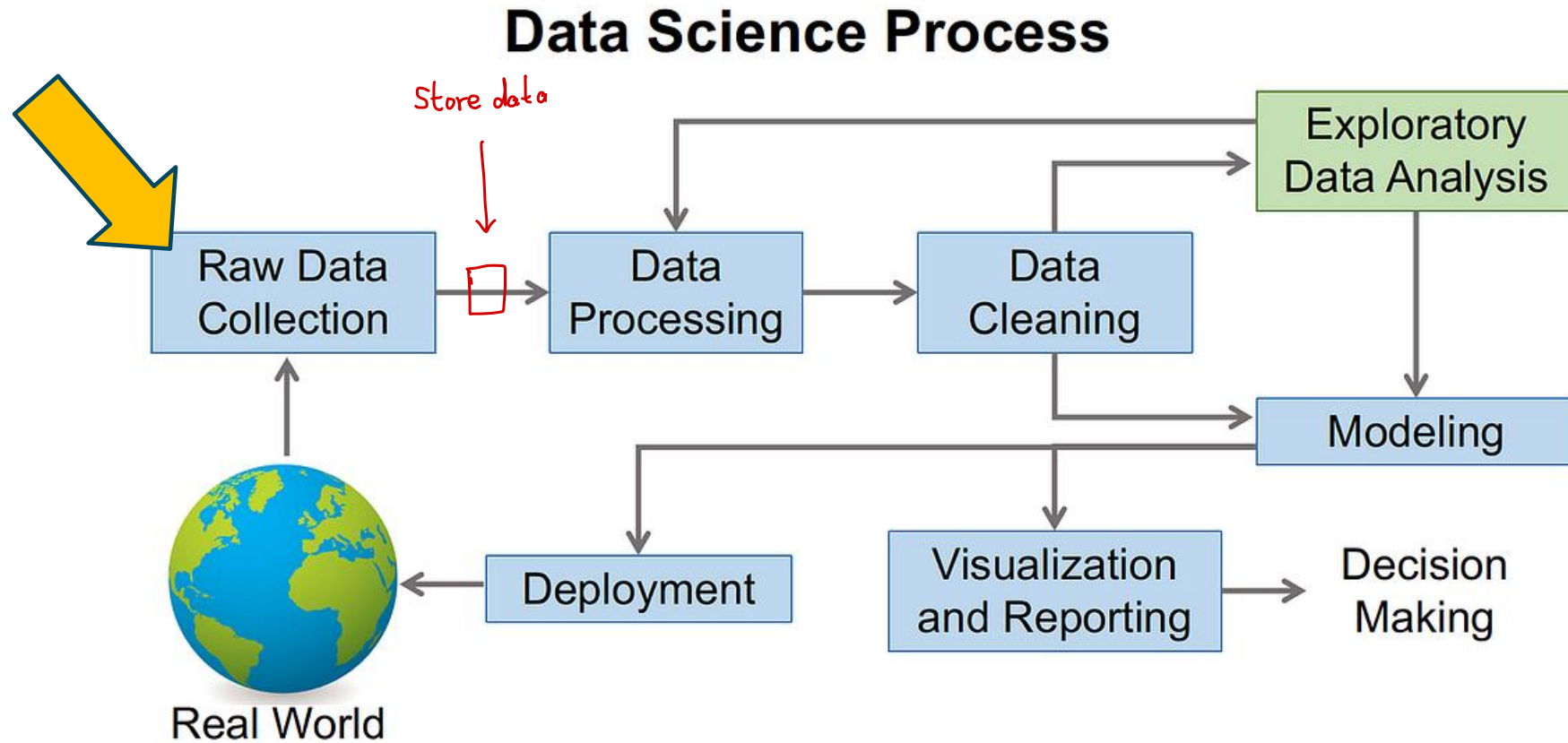
DATA STORAGE

- OLAP/OLTP
- Relational Database
- SQL vs NoSQL
- Expanding to more Data Types and Data Storage

A for Analytical

T for Transaction

OLAP / OLTP



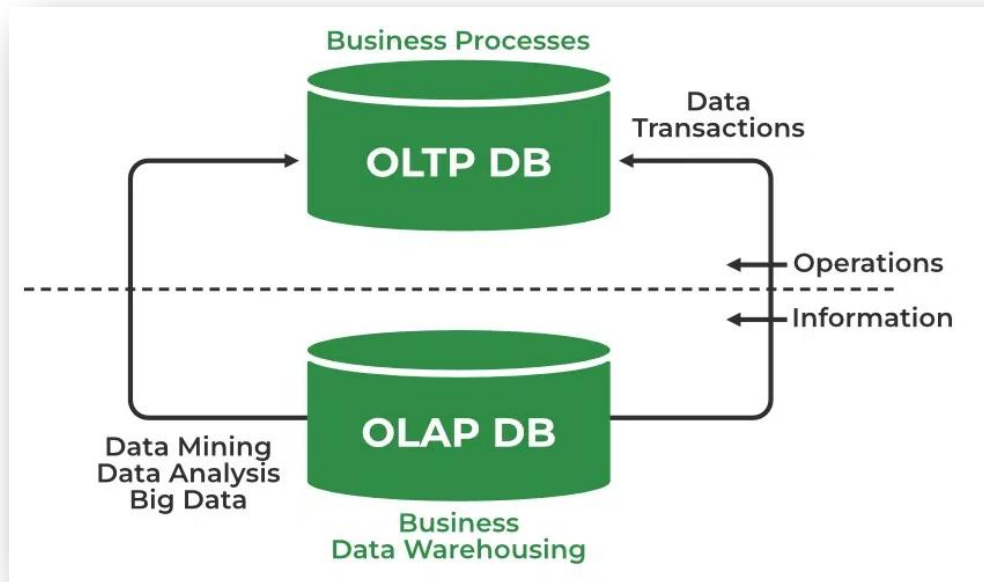
Self-Reported Data	Surveys Interviews Focus groups
Data directly provided by individuals.	
Behavioral and Observational Data	Observations Experiments
Data collected through monitoring or observing human behavior.	
Automated Data	Logs Sensor Transactions
Data generated obtained through an automated system.	
Digital and Platform-Based Data	Social Media Data Web Scraping APIs
Data originating from digital platforms, systems, or online activity.	

Sources

Online Analytical
Processing (OLAP)

Data analytics at
large scale

Analyze large amount
of data



◆ OLTP (Online Transaction Processing)

Systems that handle day-to-day transactional operations.

- Focus: **fast inserts, updates, deletes**
- Example: Recording a new food order, updating payment status
- Characteristics: Many small, short transactions, high concurrency, strict ACID compliance

Transactional

Transactional Data

◆ OLAP (Online Analytical Processing)

Systems that analyze large volumes of historical data to support decision-making.

- Focus: **complex queries, aggregations, reporting**
- Example: Finding total orders per restaurant over the last month, trend analysis
- Characteristics: Fewer transactions, read-heavy, optimized for queries and aggregation

Relational Database

Other types of
databases??

OLTP vs OLAP



Transaction

- High volume of transactions
- Fast processing
- Normalized data
- Many tables
- "Who bought X?"

VS



Analytics

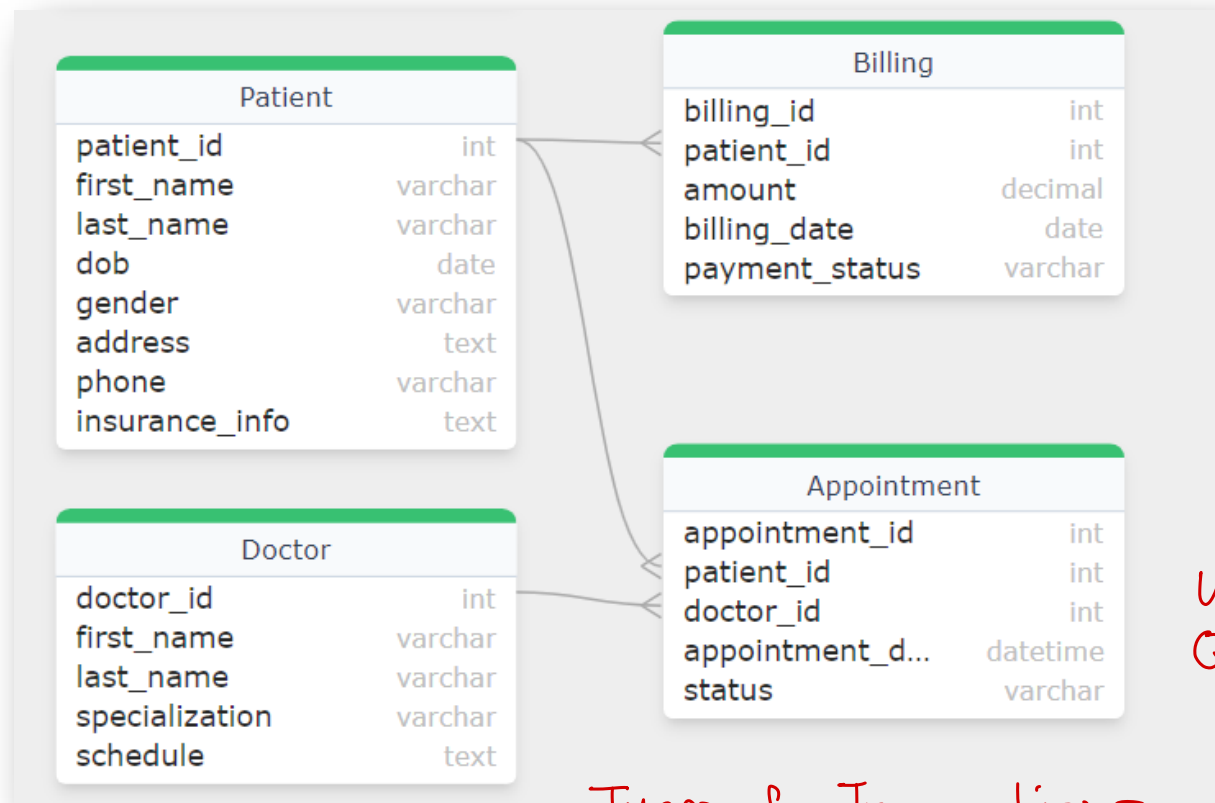
- High volume of data
- Slow queries
- Denormalized data
- Fewer tables
- "How many people bought X?"

Relational Databases

Core Principles

Core principles:

1. Data is organized in Tables
2. Schema enforces structure
3. Use of keys to define relationships
4. ACID properties for reliable transactions



Lots of transactions
that can be done

Update
Quickly

Types of Transactions

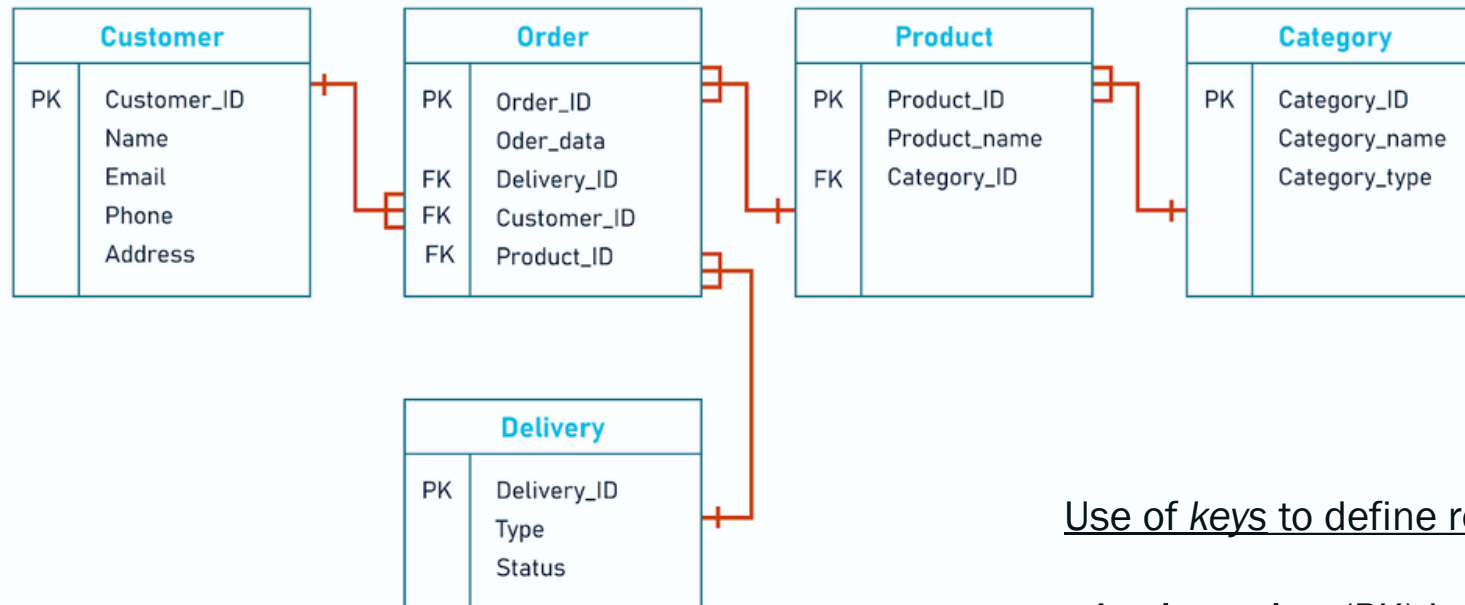
How to Design a Database for Healthcare Management System

Last Updated : 05 Mar, 2024

[Source](#)



RELATIONAL DATABASE

Use of keys to define relationships

- A **primary key** (PK) is a unique identifier for each row in a table (e.g., a Customer ID).
- A **foreign key** (FK) is a column in one table that refers to the primary key in another table, linking the data across tables.

Students

student_id	name	age
1	Alice	22
2	Bob	24

Courses

course_id	title
101	Data Science
102	AI

Enrollments

student_id	course_id
1	101
1	102
2	101

Friends

student_id_1	student_id_2
1	2



SQL is the language used for Relational Databases and developed for efficiency of transactional queries (insert, delete, update)

```
INSERT INTO Students VALUES (3, 'Charlie', 21);
INSERT INTO Enrollments VALUES (3, 102);
```

Insert

```
DELETE FROM Students WHERE student_id = 2;
```

Delete

```
UPDATE Students SET age = 23 WHERE student_id = 1;
```

Update

Databases

ACID PROPERTIES

A	Atomicity	All operations within a transaction are treated as a single unit—if one operation fails, the entire transaction fails.
C	Consistency	Transactions adhere to the rules of the database and bring it from one valid state to another.
I	Isolation	Transactions are processed independently of each other.
D	Durability	Once a transaction is completed, it will persist even in the event of a system crash.

Students

student_id	name	age
1	Alice	22
2	Bob	24

Courses

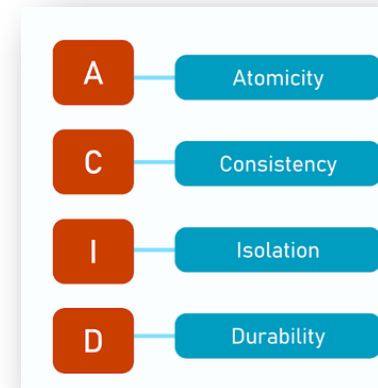
course_id	title
101	Data Science
102	AI

Enrollments

student_id	course_id
1	101
1	102
2	101

Friends

student_id_1	student_id_2
1	2



Show the names of the courses taken by each student.

Searching for data might involve multiple joins (slowing down the query).

```
SELECT s.name, c.title
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON c.course_id = e.course_id;
```

join tables

SQL / NoSQL



Source

The **importance of databases** extends across nearly all fields but is particularly critical in data science. Data science projects often involve analyzing large volumes of data to derive insights, make predictions, or inform decision-making.

For instance, traditional business applications often rely on structured data that fits well into tables with predefined schemas, making **relational databases** an ideal choice.

However, with the rise of big data, social networks, and real-time analytics, the limitations of relational databases in handling unstructured data, scaling horizontally, or managing highly connected data became evident.

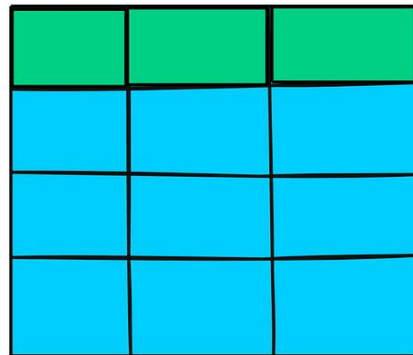


SQL vs NoSQL - 7 Key Differences You Must Know

ASHISH PRATAP SINGH
SEP 20, 2024

[Source](https://blog.algomaster.io)

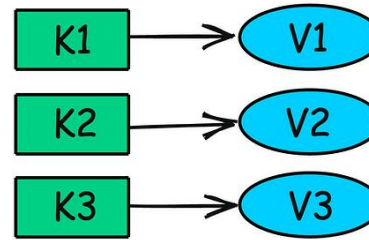
SQL



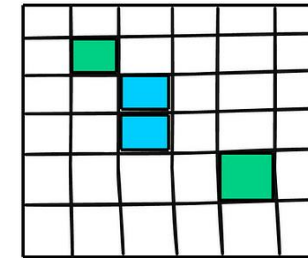
Relational

blog.algomaster.io

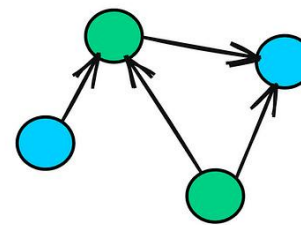
NoSQL



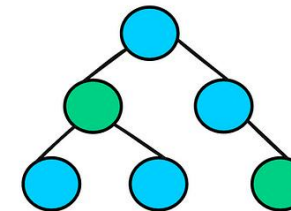
Key-Value



Column Store



Graph



Document

Most NoSQL is about how to store differently, for flexibility, efficiency, explicitness, or decreasing latency.

For different purposes



Types and Examples of NoSQL Databases

by bigdata • 24 February 2014 • Big Data, Cassandra, Cloudera, Couchbase, Hadoop, News, Hadoop

[Source](#)

The one more inline with OLAP... but let's explore all 4 types.

Key Value



Example:
Riak, Tokyo Cabinet, Redis server, Memcached, Scalaris

Document-Based



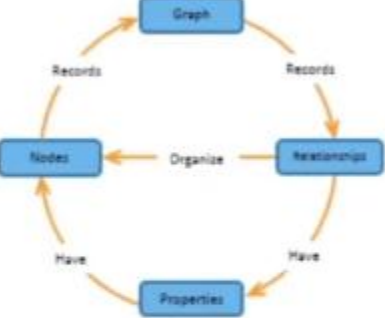
Example:
MongoDB, CouchDB, OrientDB, RavenDB

Column-Based



Example:
BigTable, Cassandra, Hbase, Hypertable

Graph-Based



Example:
Neo4J, InfoGrid, Infinite Graph, Flock DB

Key Value

Key Value



Example:
Riak, Tokyo Cabinet, Redis
server, Memcached,
Scalaris

NoSQL Databases Visually Explained with Examples

Linda Nguyen, Tech Journalist

[Source](#)

KEY-VALUE DATABASE

KEY

VALUE

K1

AAA, BBB, CCC

K2

AAA, BBB

K3

AAA, DDD

K4

AAA, 2, 01/01/2015

K5

3, ZZZ, 5623

Think of it as a light-weight quick R/W process

key: access information

can retrieve information with this key



Product ID		Type	Schema is defined per item		
Items	1	Book ID	Odyssey	Homer	1871
	2	Album ID	6 Partitas	Bach	
	2	Album ID: Track ID	Partita No. 1		
	3	Movie ID	The Kid	Drama, Comedy	Chaplin

Often used for fast changing information

Keys can lead to different information

information
Schema defined
per item

For quick change data

Product ID		Type	
Items	1	Book ID	 600
	2	Album ID	 291
	2	Album ID: Track ID	 72
	3	Movie ID	 29K

Document Based

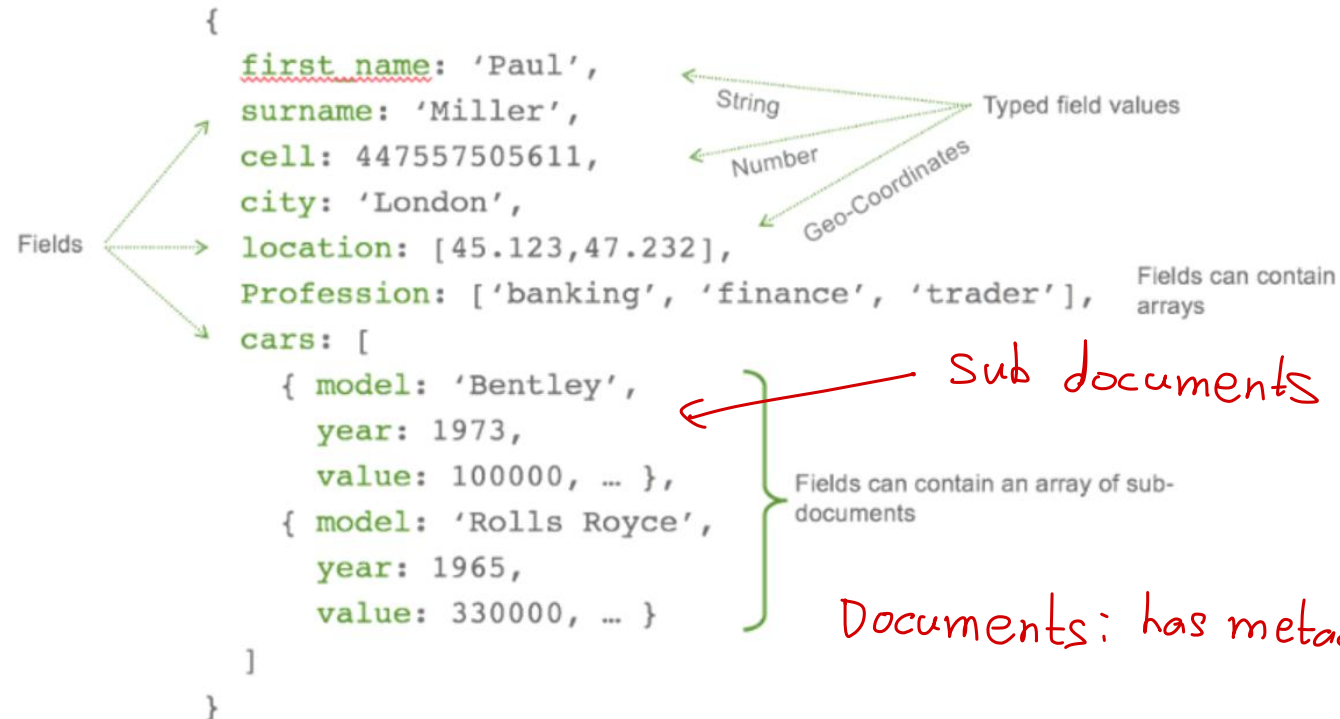
Document-Based



Example:
MongoDB, CouchDB,
OrientDB, RavenDB

Can be any type of
field

Encapsulation



Can be done in
relational databases

Documents: has metadata and content

Different view, more unified and flexible

Users Table

<u>UserId</u>	Name	Email	Age
1	John	john@email.com	28
2	Mike	mike@email.com	31
3	Ron	ron@email.com	26

Orders Table

<u>OrderId</u>	UserId	Product	Price
101	1	Laptop	\$1200
102	2	Smartphone	\$800
103	3	Headphones	\$150



```
{
  "_id": 1,
  "name": "John",
  "email": "john@email.com",
  "age": 28,
  "orders": [
    {
      "orderId": 101,
      "product": "Laptop",
      "price": 1200
    },
    {
      "orderId": 104,
      "product": "Smartphone",
      "price": 800
    }
  ]
}
```

Expands the key-value to more structured schema as values

```
{
  "_id": 1,
  "name": "John",
  "email": "john@email.com",
  "age": 28,
  "orders": [
    {
      "orderId": 101,
      "product": "Laptop",
      "price": 1200
    },
    {
      "orderId": 104,
      "product": "Smartphone",
      "price": 800
    }
  ]
}
```

Document databases allow you to query information not just by the unique identifier (like the key in key-value databases) but also by the internal content.

You can apply conditions or filters to your queries. For example, you can find all items priced more than 1000\$.

Document databases also support aggregation operations (calculating, grouping, and summarizing data) and indexing (locating and retrieving documents based on the indexed fields).

Medical Domain

json

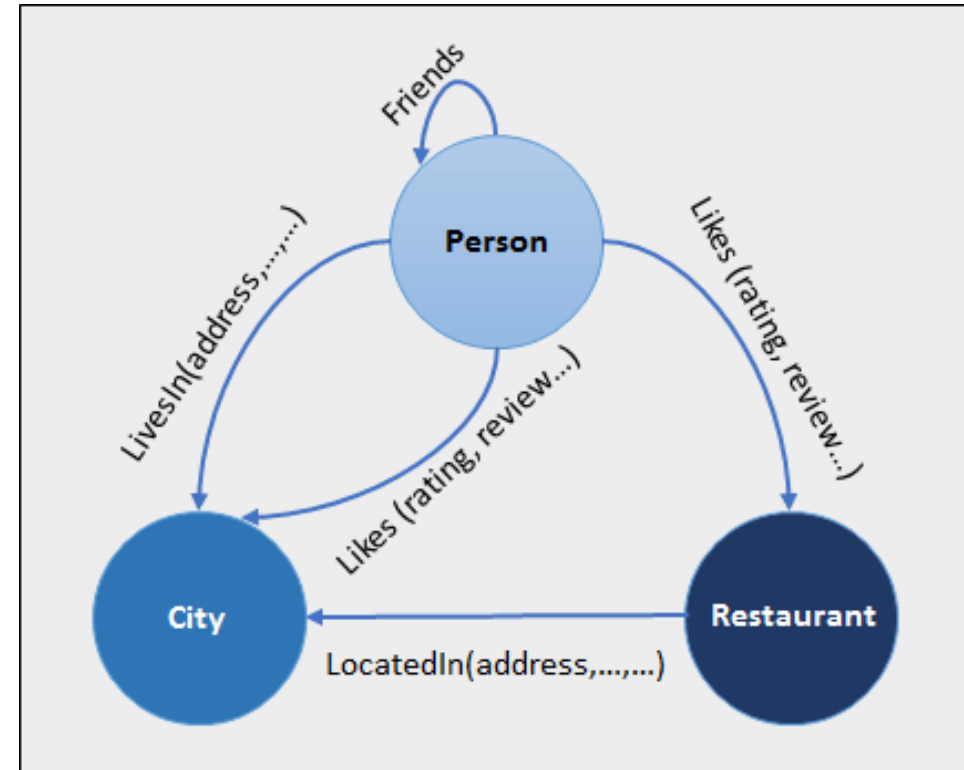
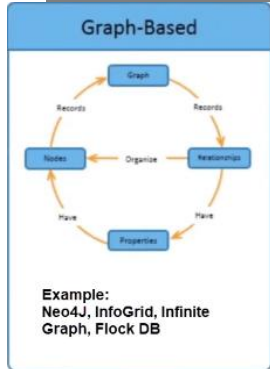
```
{
  "patient_id": "101",
  "name": "Alice Kim",
  "visits": [
    {
      "date": "2024-03-25",
      "doctor": "Dr. Smith",
      "diagnosis": "Hypertension",
      "medications": ["Lisinopril"]
    }
  ]
}
```

Education Domain

</> JSON

```
{
  "_id": 1,
  "name": "Alice",
  "age": 22,
  "courses": [
    {"course_id": 101, "title": "Data Science"},
    {"course_id": 102, "title": "AI"}
  ],
  "friends": [2]
}
{
  "_id": 2,
  "name": "Bob",
  "age": 24,
  "courses": [
    {"course_id": 101, "title": "Data Science"}
  ],
  "friends": [1]
}
```

Graph-Based



Explicitly

The main concept of a graph database system is a relationship. Data is related together in a graph to store a collection of nodes and edges, where the edges represent the relationship between nodes.

Different conceptual views

Focuses on relationships

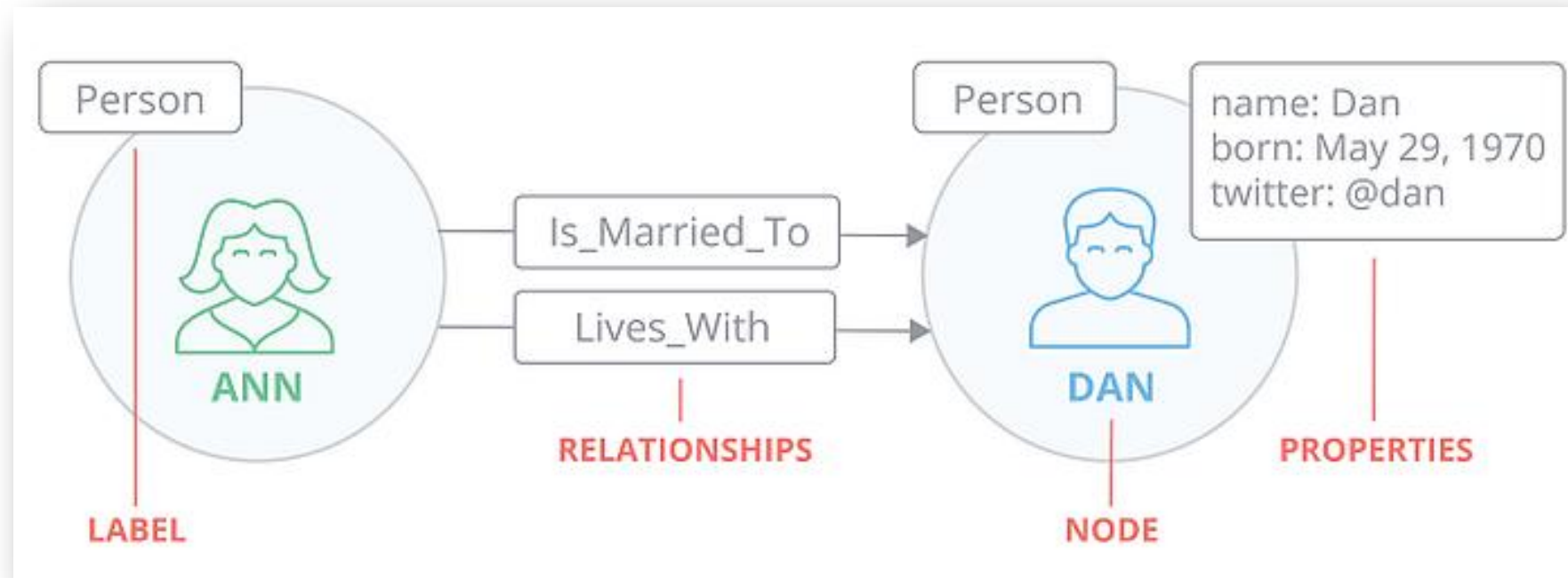
Graph database design for our social travel App



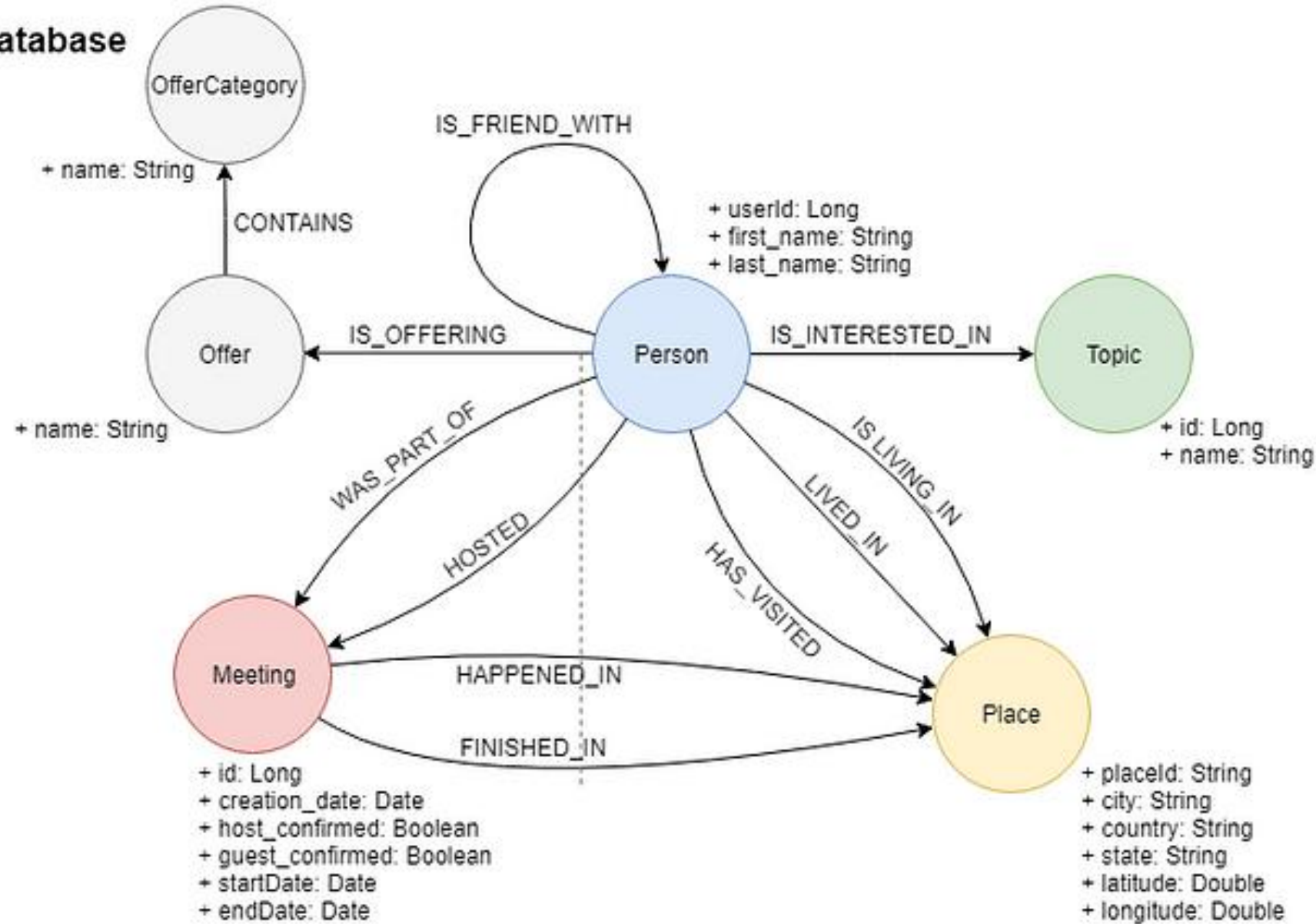
Guillaume Prévost · Follow
7 min read · Jun 14, 2018

[Source](#)

View of the data representation
as nodes and edges



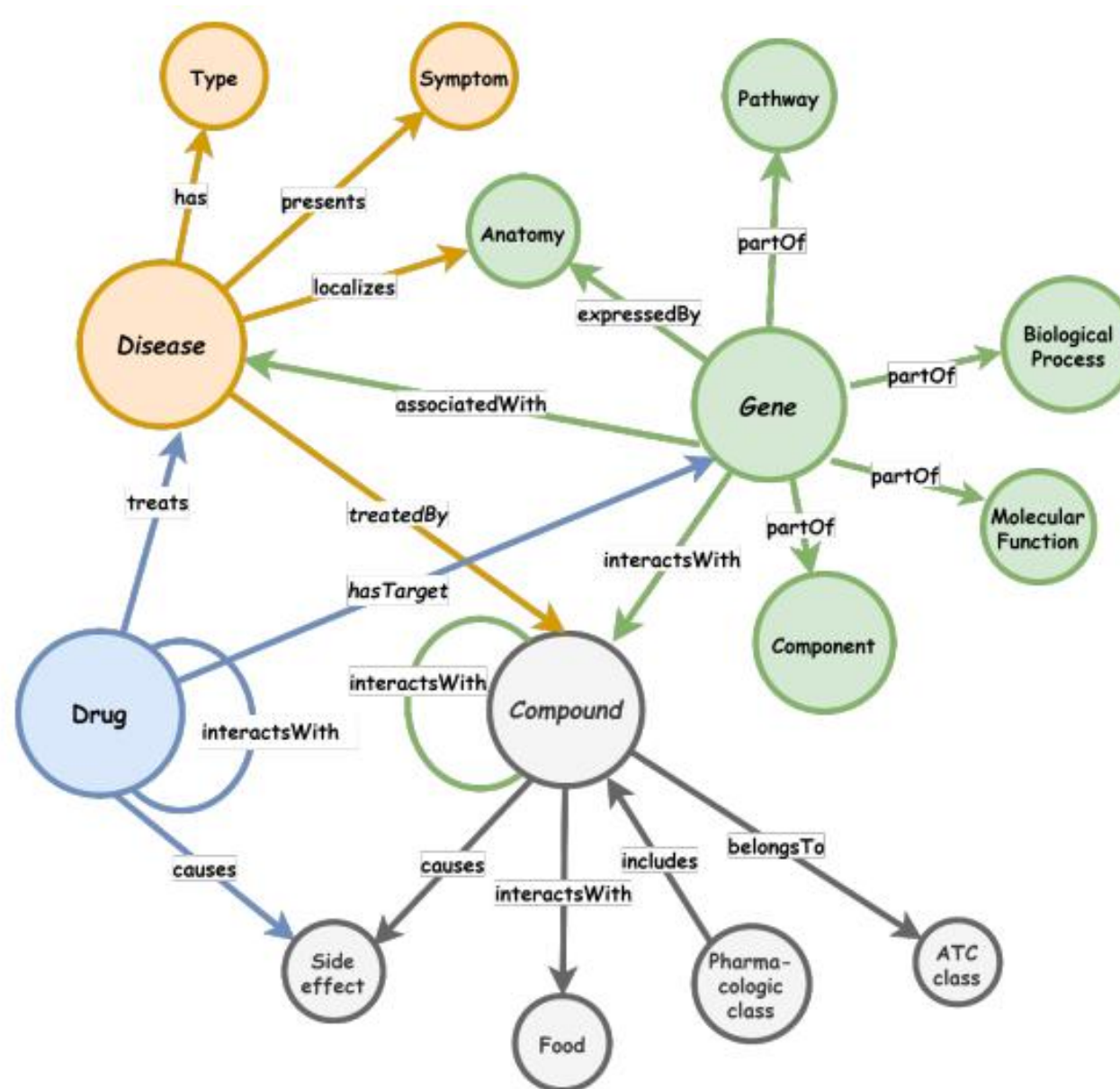
Graph Database (Neo4J)



Clinical Decision Support Using Healthcare Knowledge Graphs

Haziqa Sajid | December 28, 2022

[Source](#)



Cypher (Graph Database Query Language)

Concept	Example
Node	<code>(u:User {name: "Alice"})</code> → a user node named Alice
Relationship	<code>(u)-[:FRIEND]->(v)</code> → Alice is friends with Bob
Property	<code>age: 30</code> → attribute of a node or relationship
Pattern Matching	<code>MATCH (u:User)-[:ORDERED]->(r:Restaurant)</code> → find users who ordered from restaurants

```
</> cypher
```

```
(Alice:Student {id:1, age:22})  
(Bob:Student {id:2, age:24})
```

```
(DataScience:Course {id:101})  
(AI:Course {id:102})
```

```
(Alice)-[:ENROLLED_IN]->(DataScience)  
(Alice)-[:ENROLLED_IN]->(AI)  
(Bob)-[:ENROLLED_IN]->(DataScience)
```

```
(Alice)-[:FRIEND]->(Bob)  
(Bob)-[:FRIEND]->(Alice)
```

Insert

```
CREATE (c:Student {id:3, name:"Charlie", age:21})  
CREATE (c)-[:ENROLLED_IN]->(:Course {id:102});
```

Delete

```
MATCH (b:Student {id:2})  
DETACH DELETE b;
```

Update

```
MATCH (a:Student {id:1})  
SET a.age = 23;
```

Find the courses in which the friends of Alice are enrolled in.

```
MATCH (a:Student)-[:FRIEND]->(f)-[:ENROLLED_IN]->(c)  
WHERE a.name = "Alice"  
RETURN f.name, c.title;
```

Column Based

Column-Based



Example:
BigTable, Cassandra,
Hbase,
Hypertable

- Intuitive idea... store each column separately
 - Faster single column analytics (e.g. summary statistics)
 - Flexibility to expand to more columns

Mar 3, 2023 · 8 min read

NoSQL wide-column stores demystified

Alex Williams

[Source](#)

Key	Fname	Lname	State	Zip	Phone	Age	Sex
1	Bugs	Bunny	NY	11217	(718) 938-3235	34	M
2	Yosemite	Sam	CA	95389	(209) 375-6572	52	M
3	Daffy	Duck	NY	10013	(212) 227-1810	35	M
4	Elmer	Fudd	ME	04578	(207) 882-7323	43	M
5	Witch	Hazel	MA	01970	(978) 744-0991	57	F



Think of all the analysis we have done so far... descriptive, predictive

Overall, column-based databases work well for **OLAP (Online Analytical Processing)** but not well for OLTP (Online Transactional Processing).

NOSQL DATABASE TYPES

	Key-value	Document	Graph	Column-oriented
Data model	Simple key-value pairs (key: value)	JSON, BSON, or similar format, with hierarchical or nested structures	Nodes or vertices (entities) and edges (relationships)	Columns grouped into column families, where each row can have different column structures
Query capabilities	Simple lookups based on keys; minimal querying functionality	Rich querying with filtering, aggregation, and indexing	Complex queries traversing relationships between nodes	Optimized for fast aggregations and analytical queries over columns
Use cases	• Performance monitoring • Storing basic information • Session management • Caching • Video game boards • IoT data	• Content Management Systems (CMS) • User profiles • Cataloging	• Social networks • Recommender systems • Fraud detection • Natural language processing	• IoT systems • Big data analytics • BI tools
Most popular databases	• Redis • Memcached • RocksDB • Hazelcast • Valkey	• MongoDB • Apache CouchDB • Firebase Firestore • Amazon DocumentDB	• Neo4j • Amazon Neptune • JanusGraph	• Apache Cassandra • Apache HBase • Apache Druid • ScyllaDB

More database types

Database Management Systems

SHEKHAR JHA / FEBRUARY 19, 2025 / OTHER DATABASE PLATFORMS

Problem

Data can be in the form of text, numbers, multimedia, models, programming languages, and other special formats depending on the field. Also, information may be analog or digital, as well as quantitative, nominal, ordinal, or continuous. And now, in this digital era, the challenge is: how should we store and consume data to gain value from it? Let's take a look at different database management systems.

We will cover the following:

- [The ACID Model in Databases](#)
- [A Taxonomy of Database Systems](#)
- [Relational/SQL Databases](#)
- [NoSQL Database](#)
- [Key-Value Store](#)
- [Document Databases](#)
- [Graph Database](#)
- [Wide-Column Stores](#)
- [In-Memory Databases](#)
- [Time-Series Databases](#)
- [Object-Oriented Databases](#)
- [Text Search Databases](#)
- [Blob Datastore](#)
- [Spatial Databases](#)
- [Vector Databases](#)

15 Types of Databases and When to Use Them

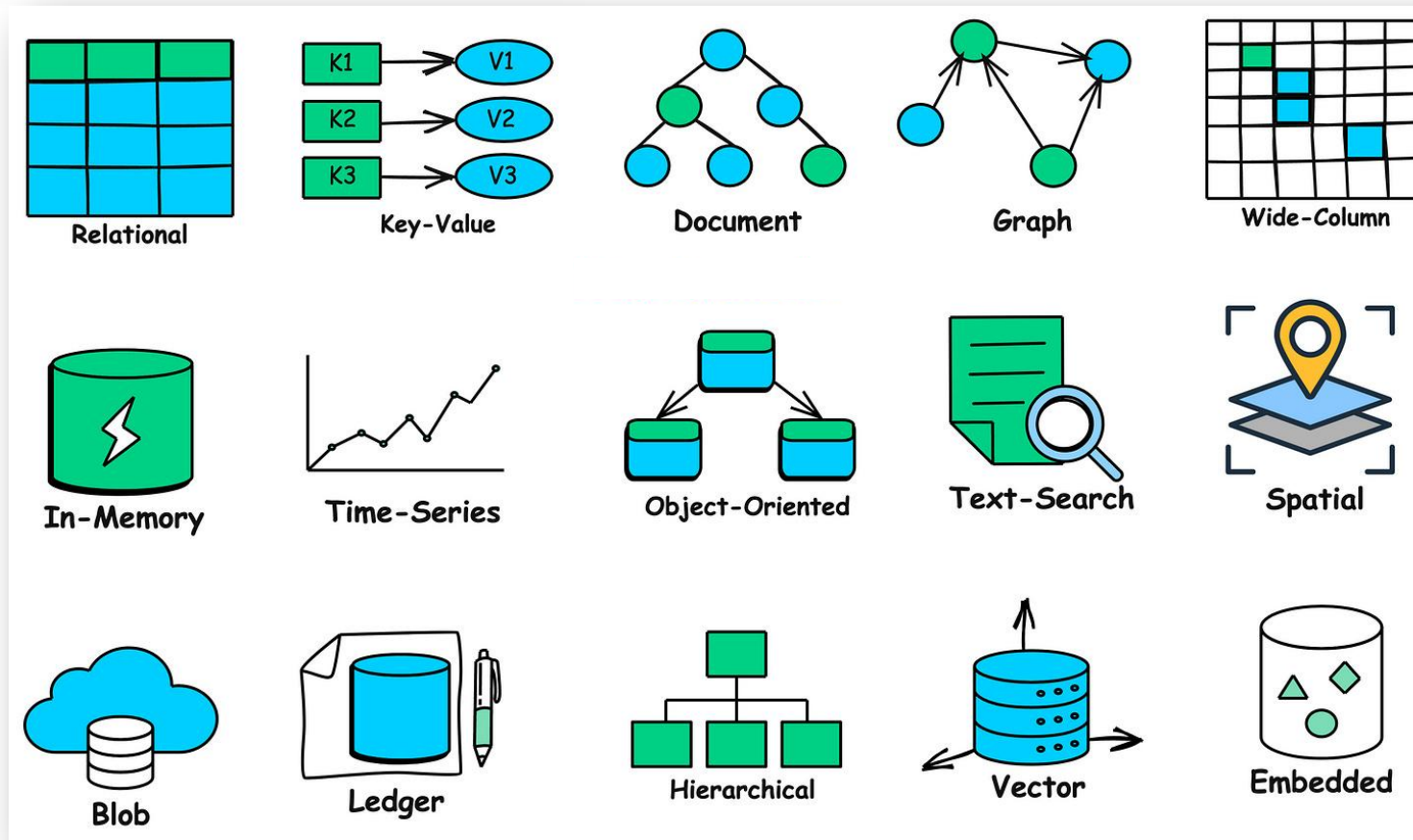
#4 System Design - Types of Databases

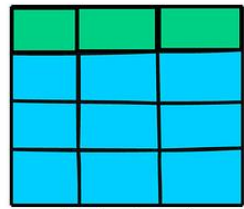


ASHISH PRATAP SINGH
MAR 24, 2024

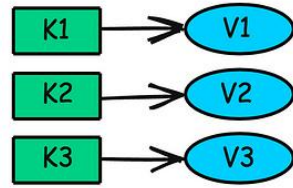
A bit overwhelming when
seen like this

Source

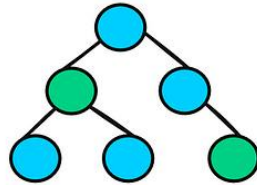




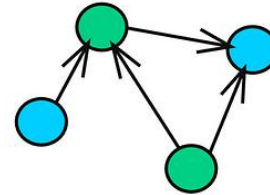
Relational



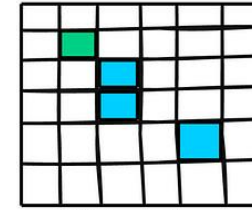
Key-Value



Document

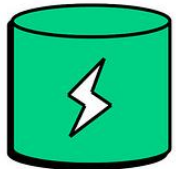


Graph



Wide-Column

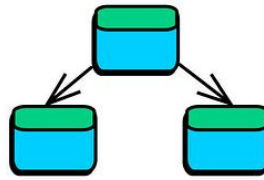
We've already went through these 5



In-Memory



Time-Series



Object-Oriented



Text-Search



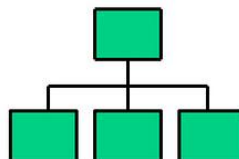
Spatial



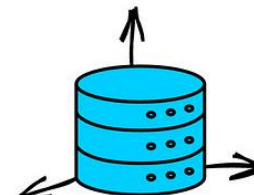
Blob



Ledger



Hierarchical



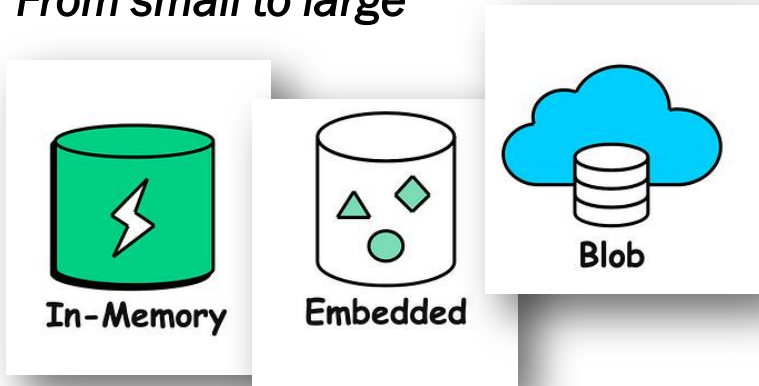
Vector



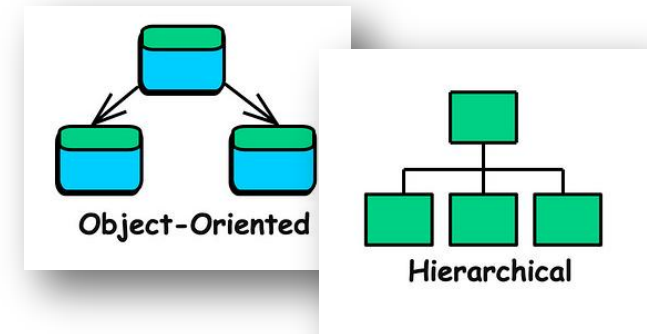
Embedded

Let me try to reorganize this list

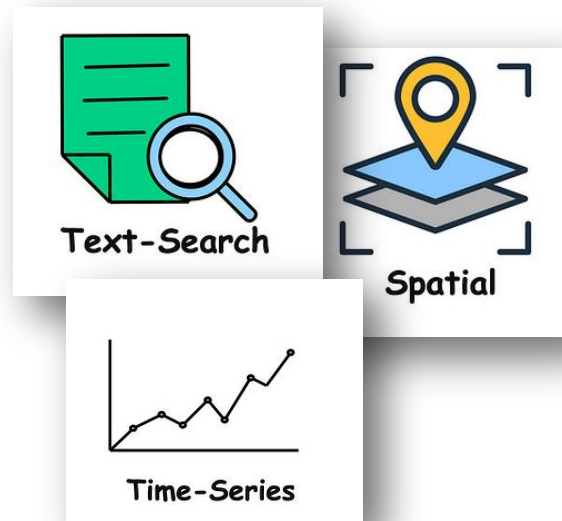
From small to large



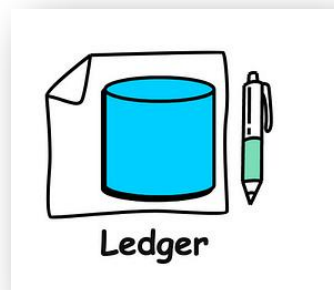
The necessary fails



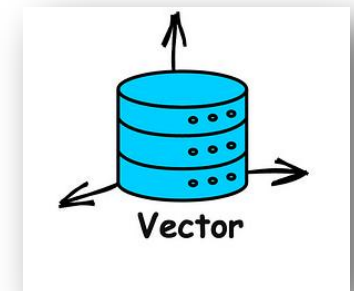
The data specific



The super secure and immutable



The newest AI trendy one



From small to large

In-Memory Databases

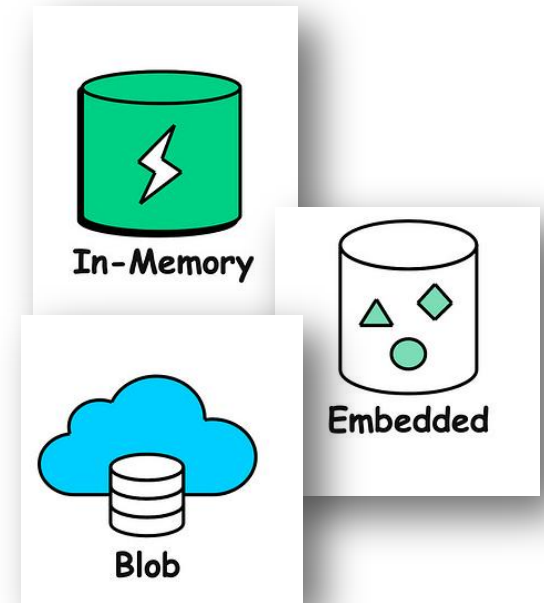
- Stored directly in the main memory (RAM) of the computer
- Used for: (very fast) online gaming

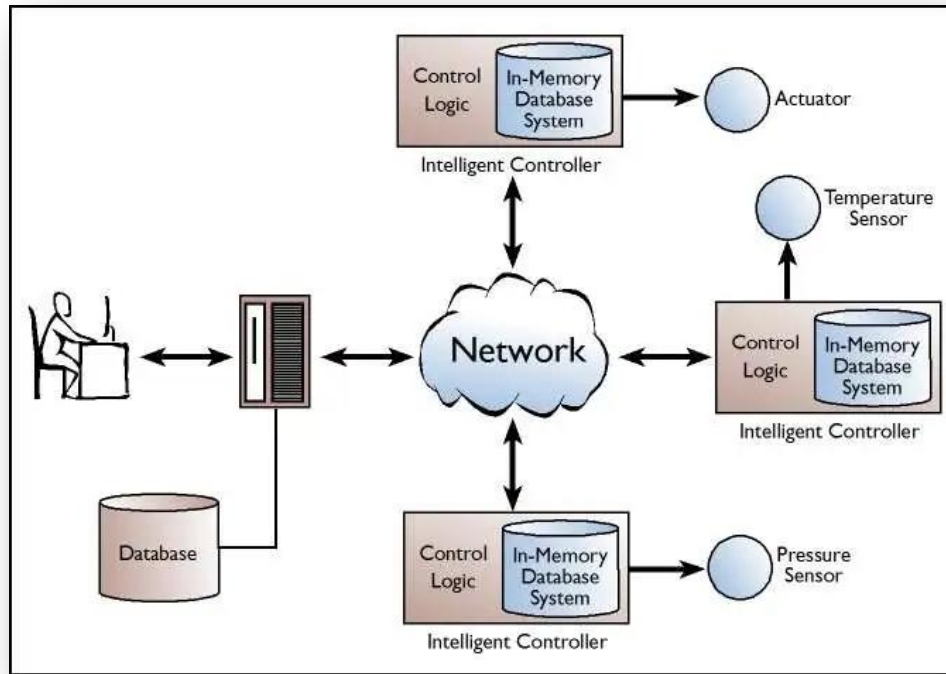
Embedded Databases

- Integrated within the software application (not run as client-server)
- Used for : desktop applications (local settings)

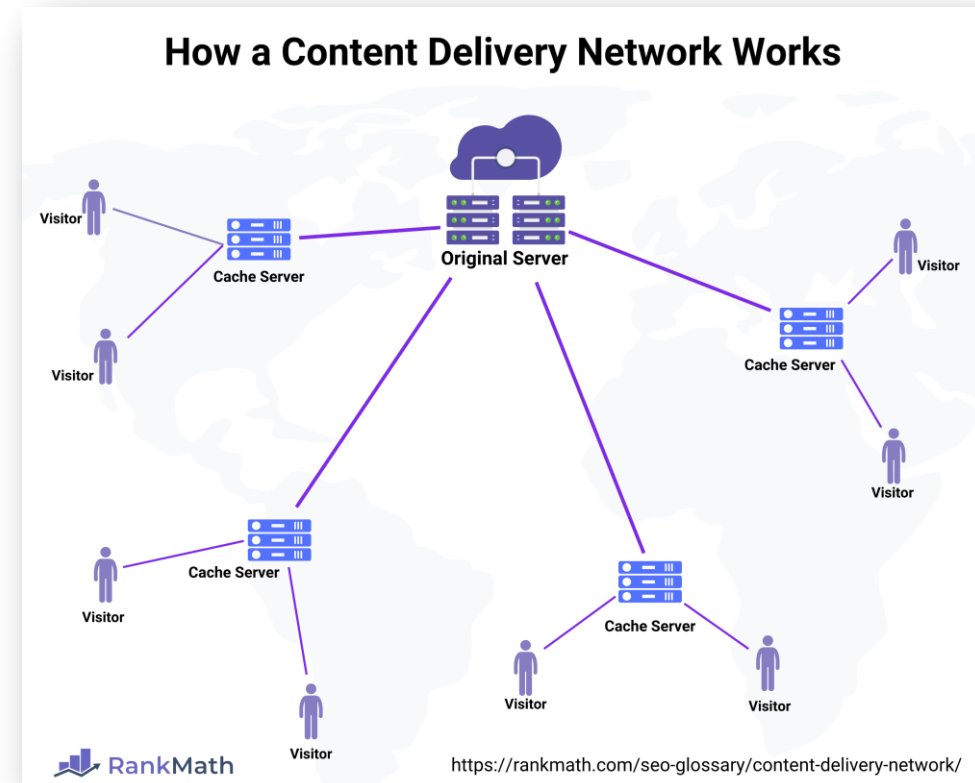
Blob Databases (Binary Large Object)

- Designed for storing, managing, and retrieving large blocks of unstructured data, such as images, audio files, videos, and documents.
- Used for: Content Delivery Networks (large media files), backups, big data storage





Memory-Based DB

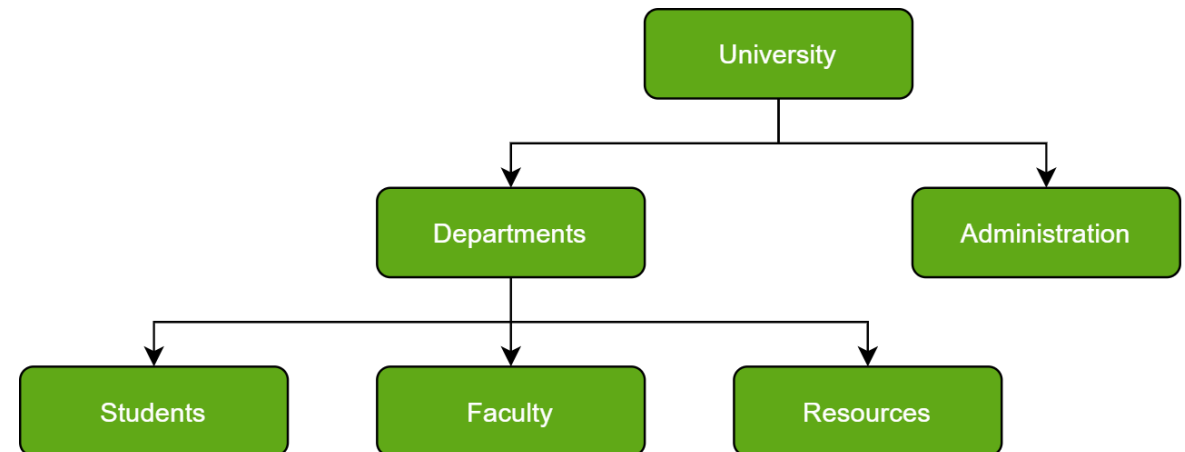
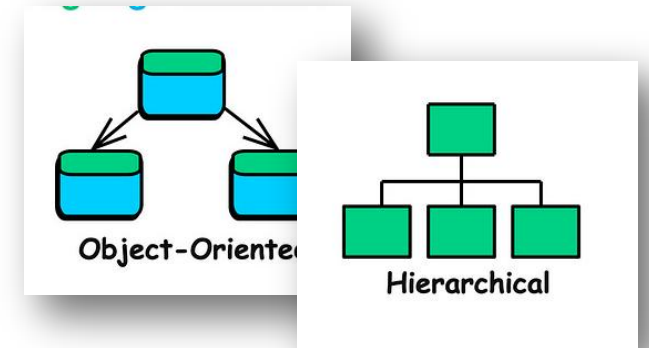


Blob DB

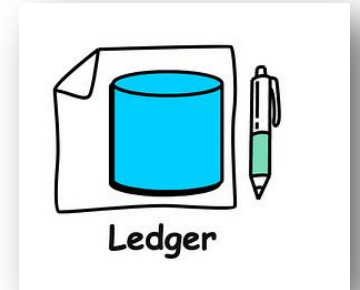
The necessary fails

Object-oriented databases (OODBs) store data as objects, similar to object-oriented programming.

Hierarchical databases organize data in a tree-like structure, with parent-child relationships between records.



The super secure and immutable



Ledger databases are designed to provide an immutable, append-only record of transactions.

They are engineered to ensure that once a transaction is recorded, it cannot be altered or deleted, providing a verifiable and tamper-evident history of all changes over time.

Used for:

- **Healthcare:** Managing patient records, consent forms, and treatment histories with a clear, unchangeable record.
- **Voting systems:** Ledger databases can provide a secure and transparent platform for conducting voting processes.

The data specific

Text Search Databases are specialized systems designed for efficient storage, indexing, and retrieval of large volumes of unstructured or semi-structured text data (e.g. logs, enterprise data). They provide fast and scalable search capabilities.

Spatial databases are designed to store, manage, and analyze data that represents geographical or spatial information. They extend traditional database capabilities to handle complex spatial data types, such as points, lines, polygons, and other geometric shapes. They are useful in different applications related to transportation, logistic, etc.

Time-series databases are optimized for storing and analyzing time-stamped data, such as sensor readings, stock prices, or server logs.

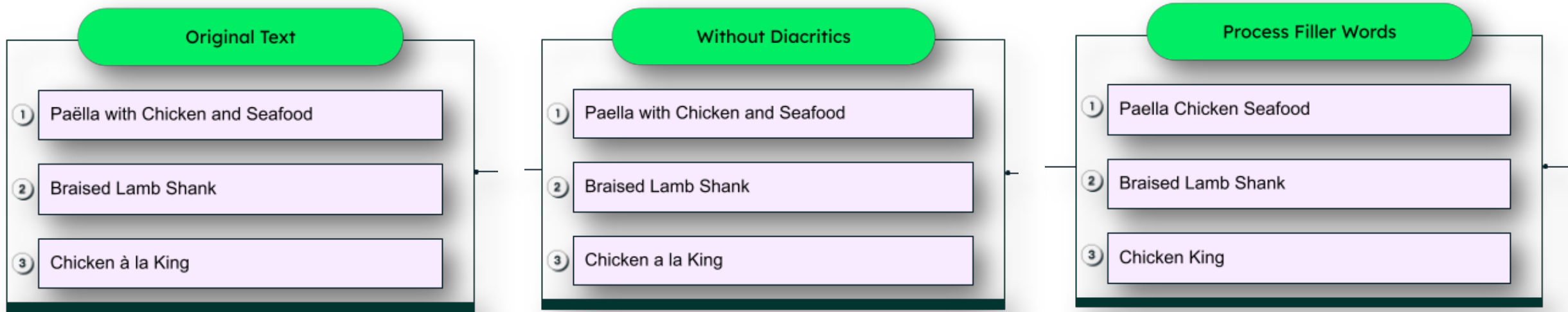




What is Full-Text Search and How Does it Work?

[Source](#)

Explains text-specific processing and indexing





What is Full-Text Search and How Does it Work?

[Source](#)



Dictionary

① PAELLA CHICKEN SEAFOOD

② BRAISE LAMB SHANK

③ CHICKEN KING

Index

BRAISE ②

CHICKEN ① ③

KING ③

LAMB ②

PAELLA ①

SEAFOOD ①

SHANK ②

Different types of search

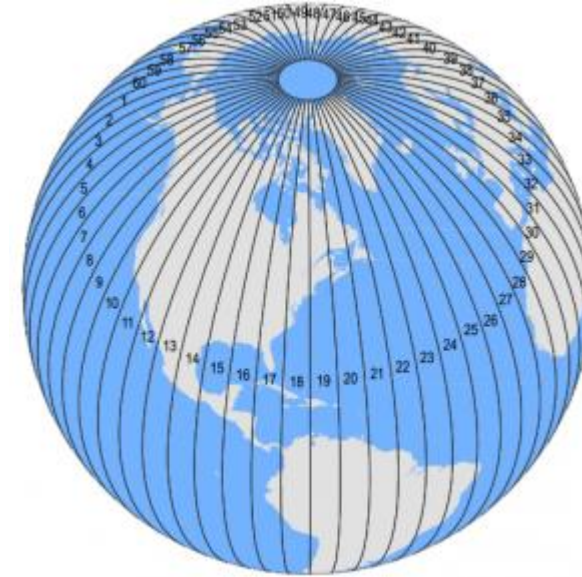
Text Search Flexibility

- Boolean search (using AND, OR, NOT)
- Fuzzy search (typos, misspellings)
- Wildcard search (e.g. part*)
- Phrase search
- Proximity search
- Range search (e.g. dates, currency)

Knowledge of
coordinate systems

GISGeography

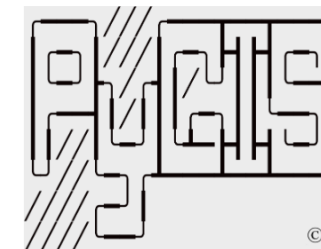
<https://gisgeography.com/utm-universal-transverse-mercator-projection/>



A UTM zone is a 6° segment of the Earth. Because a circle has 360°, this means that there are 60 UTM zones on Earth.

Knowledge of shapes
and spatial relations

https://pygis.io/docs/e_spatial_joins.html

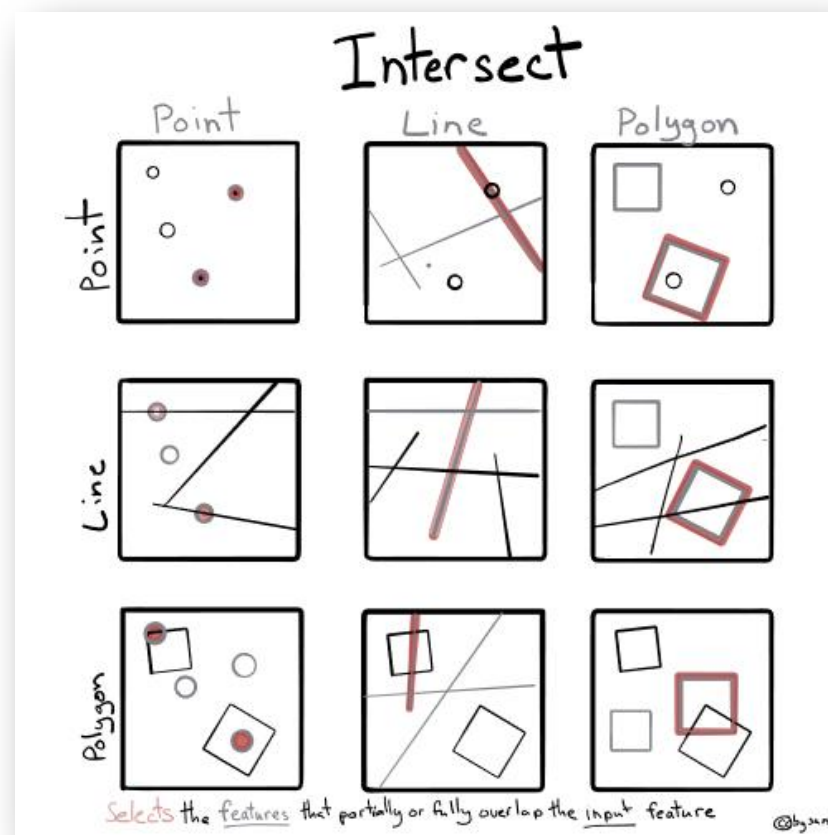


Q Search Ctrl + K

PyGIS - Open Source Spatial
Programming & Remote Sensing

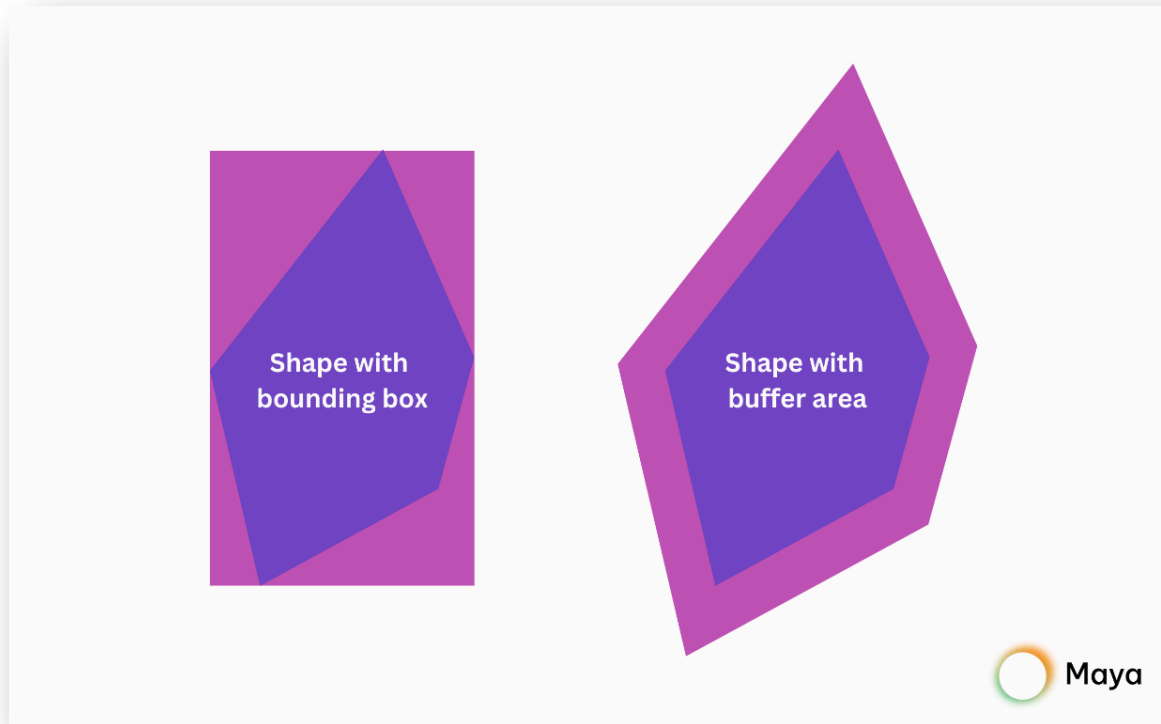
Geometry Type	
	POINT
	LINestring
	POLYGON

Contains
Intersect
Crosses
Overlap



A bounding box, and a buffer are both spatial concepts used to define specific areas surrounding a shape of interest, but they serve different purposes and have distinct characteristics.

Knowledge of shapes extensions



Weather
Table

	Tags	Fields	
Timestamp	Location	Wind Speed	Temperature
9:00	New York	3.2	20.1
9:00	London	3.6	15.2
9:05	New York	3.3	20.3
9:05	London	3.4	15.2
9:10	New York	3.4	20.5
9:10	London	3.0	15.5

Could be stored for easier
time processing



Row view / Column view

Features:

- Interpolation
- Aggregation
- Decomposition

Weather, Location=New York, Wind Speed

9:00	3.2
9:05	3.3
9:10	3.4

Weather, Location=New York, Temperature

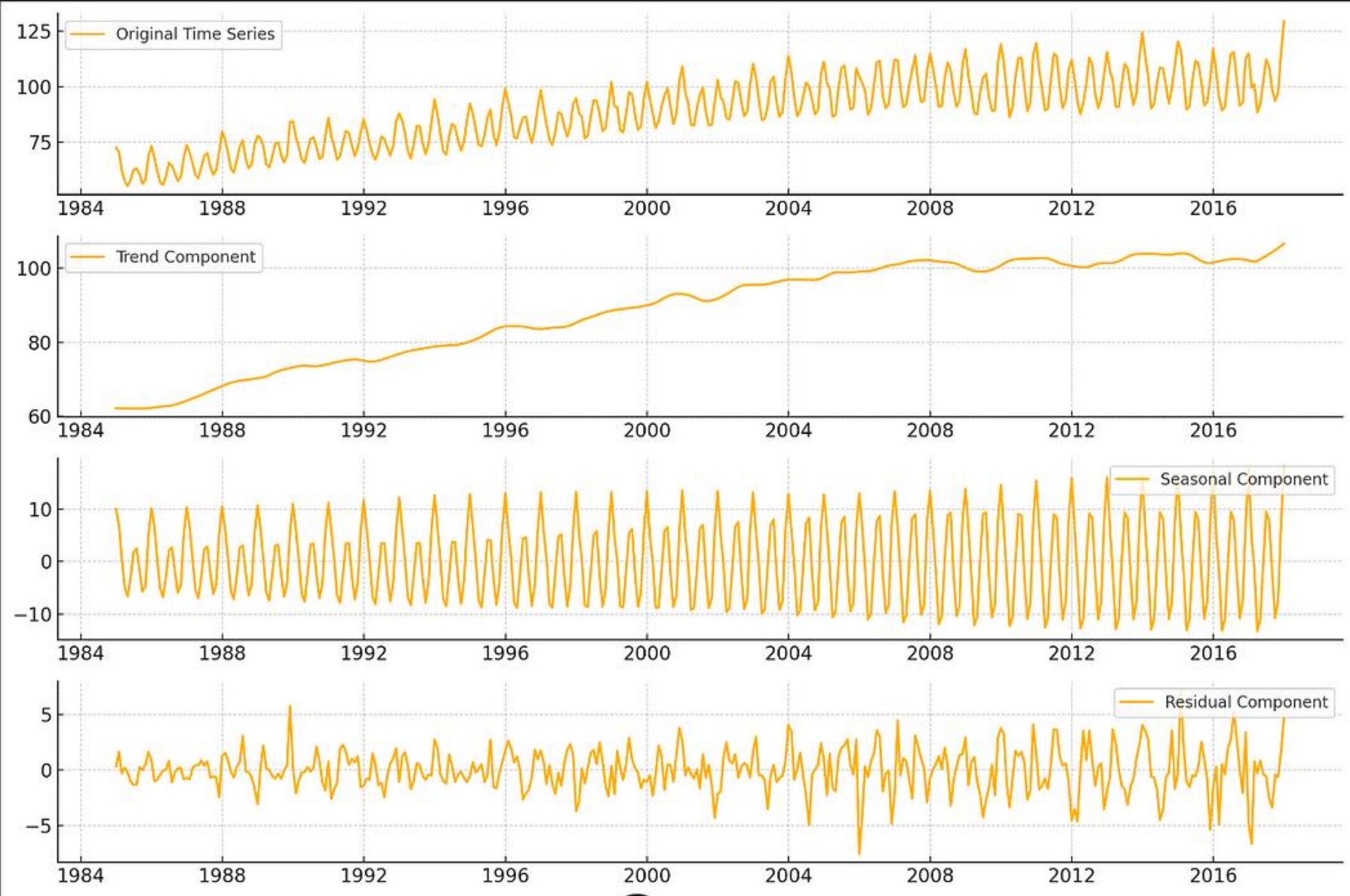
9:00	20.1
9:05	20.3
9:10	20.5

Weather, Location=London, Wind Speed

9:00	3.6
9:05	3.4
9:10	3.0

Weather, Location=London, Temperature

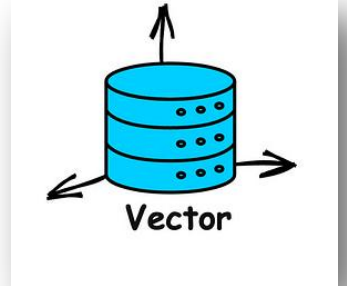
9:00	15.2
9:05	15.2
9:10	15.5

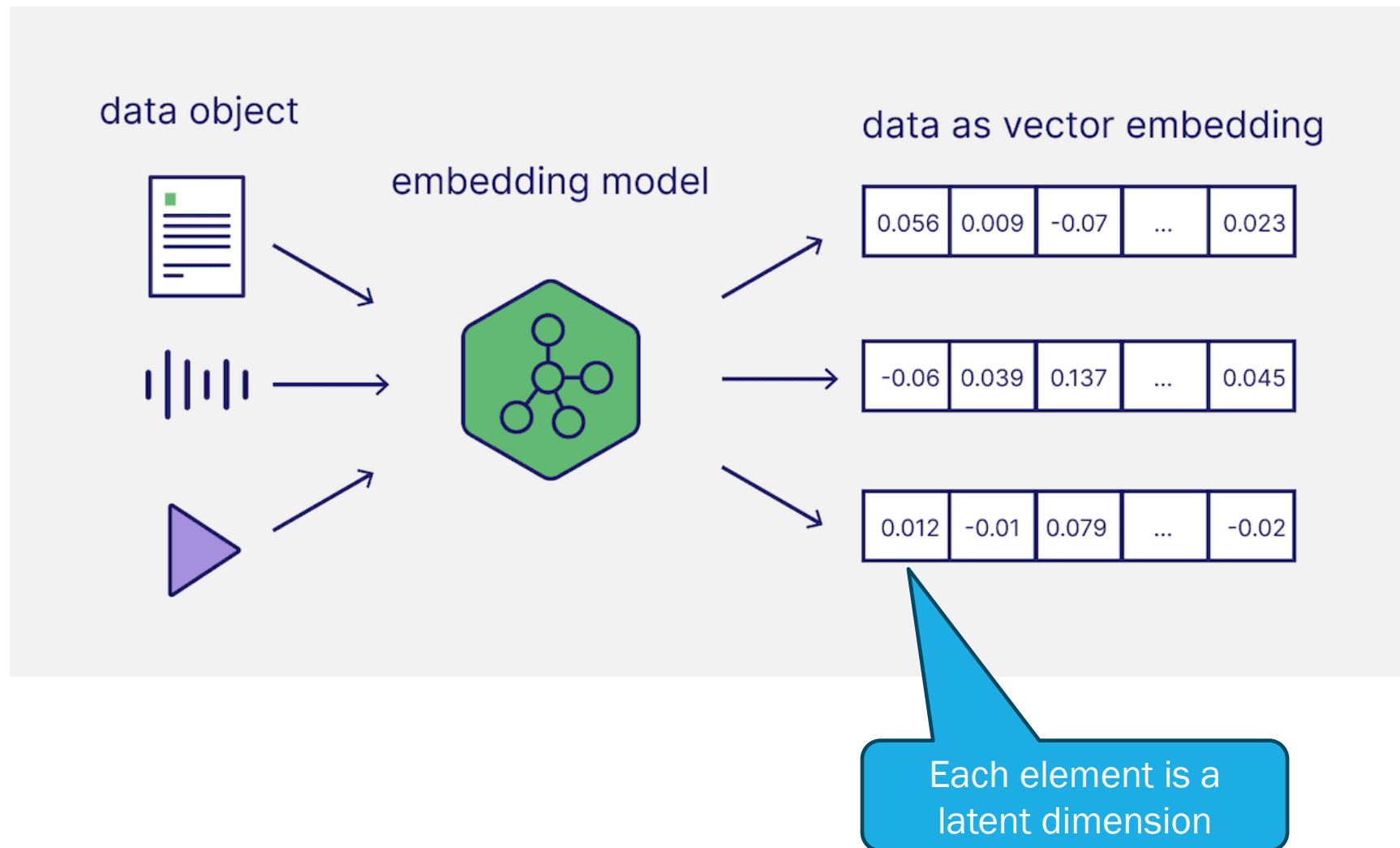


The newest AI trendy one

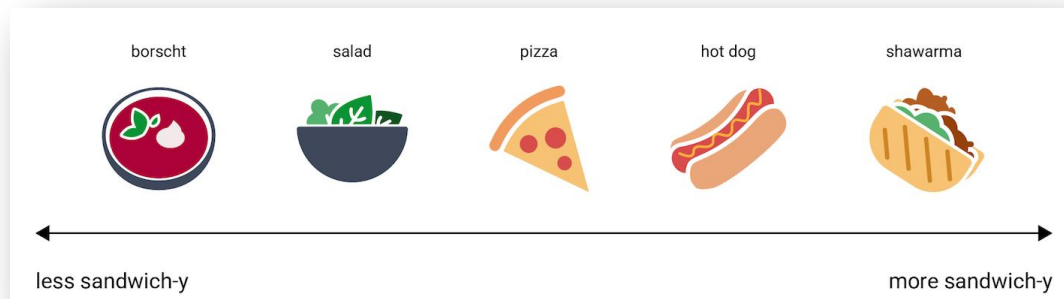
Vector databases are specialized databases designed for storing and searching vectors which are arrays of numbers representing data in high-dimensional spaces. They are optimized for similarity search and nearest neighbor queries enabling fast retrieval of similar items based on their vector representations.

These databases are particularly relevant in the field of machine learning and artificial intelligence (AI), where vector representations are commonly used to encode the features of various types of data, including text, images, and audio.

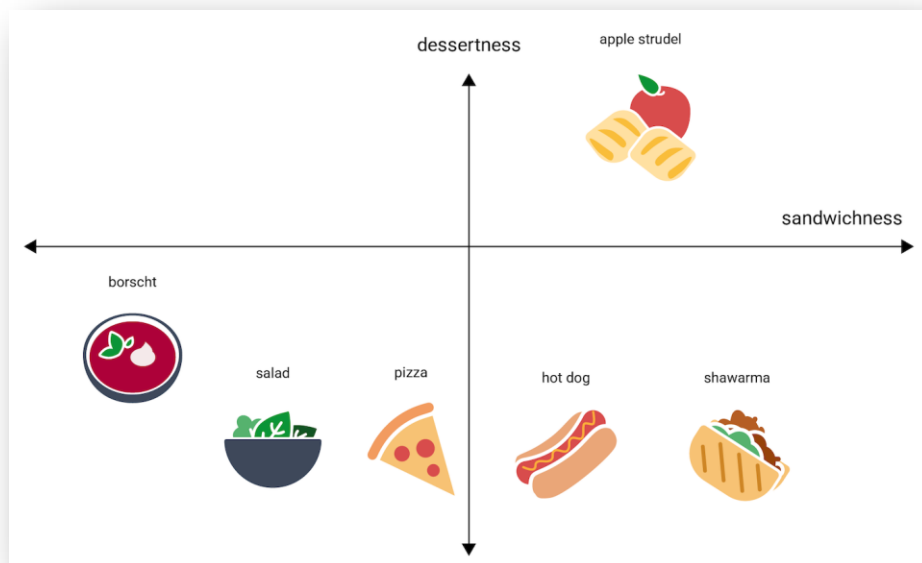




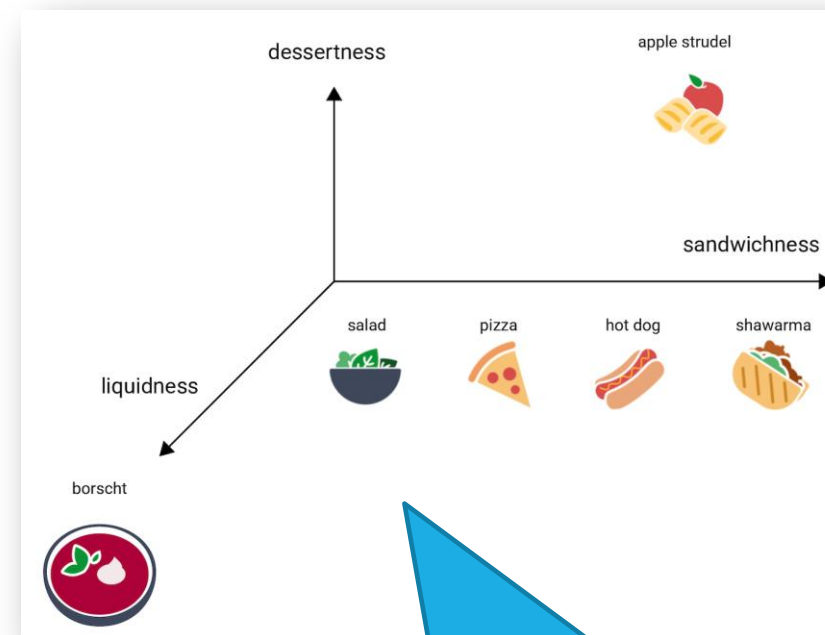
Imagine placing different food on a first dimension



Adding a second dimension

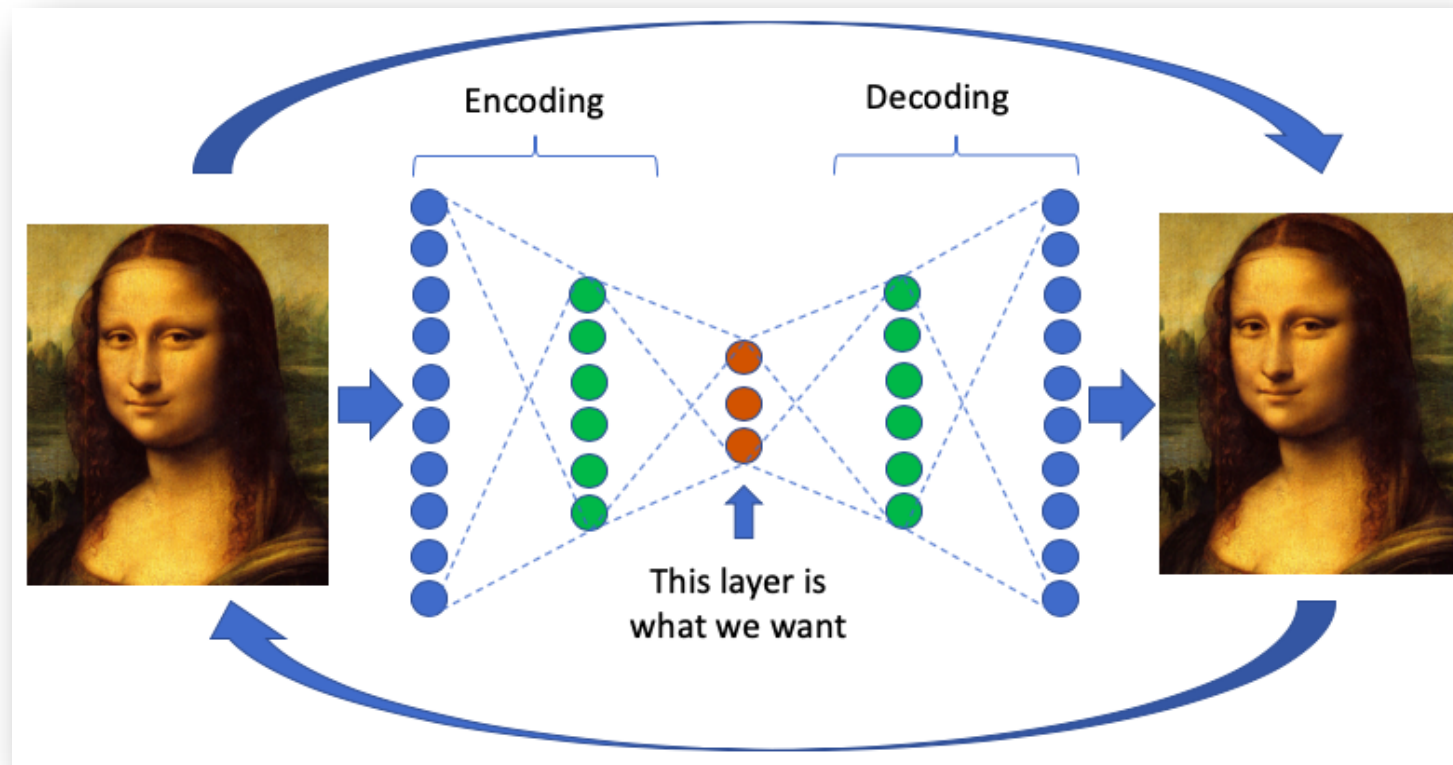


Adding a third dimension

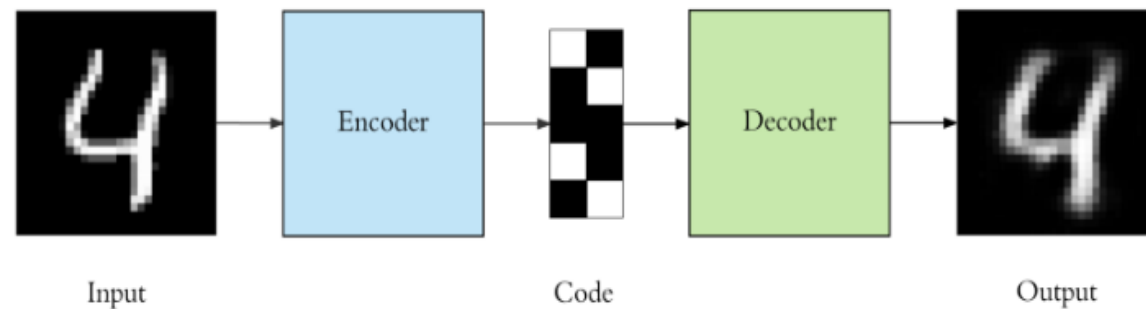


If « latent », the dimensions are automatically discovered (and NOT named)

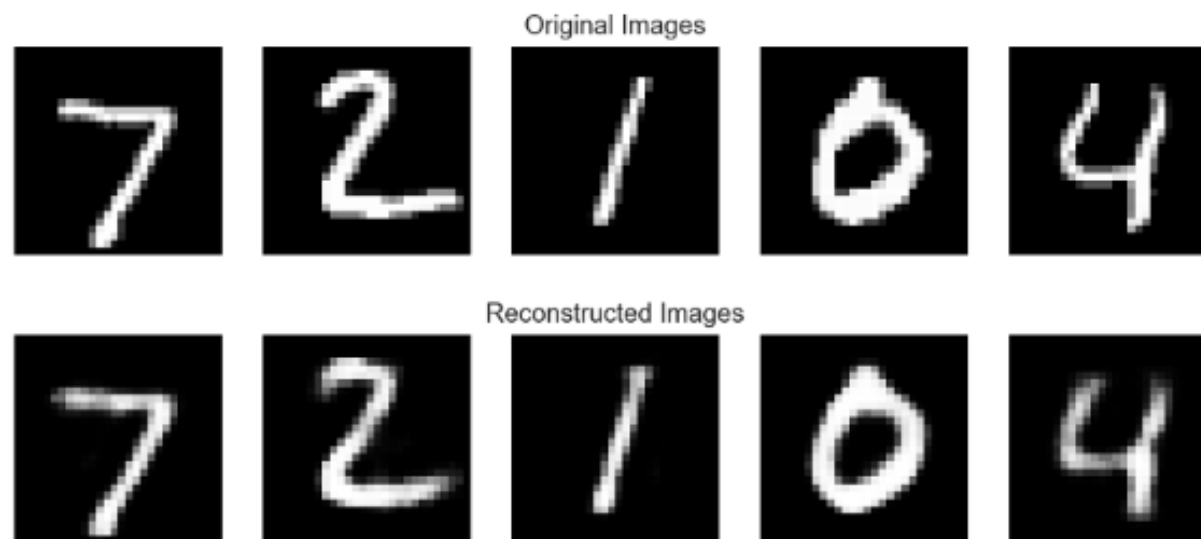
What Are Image Embeddings?



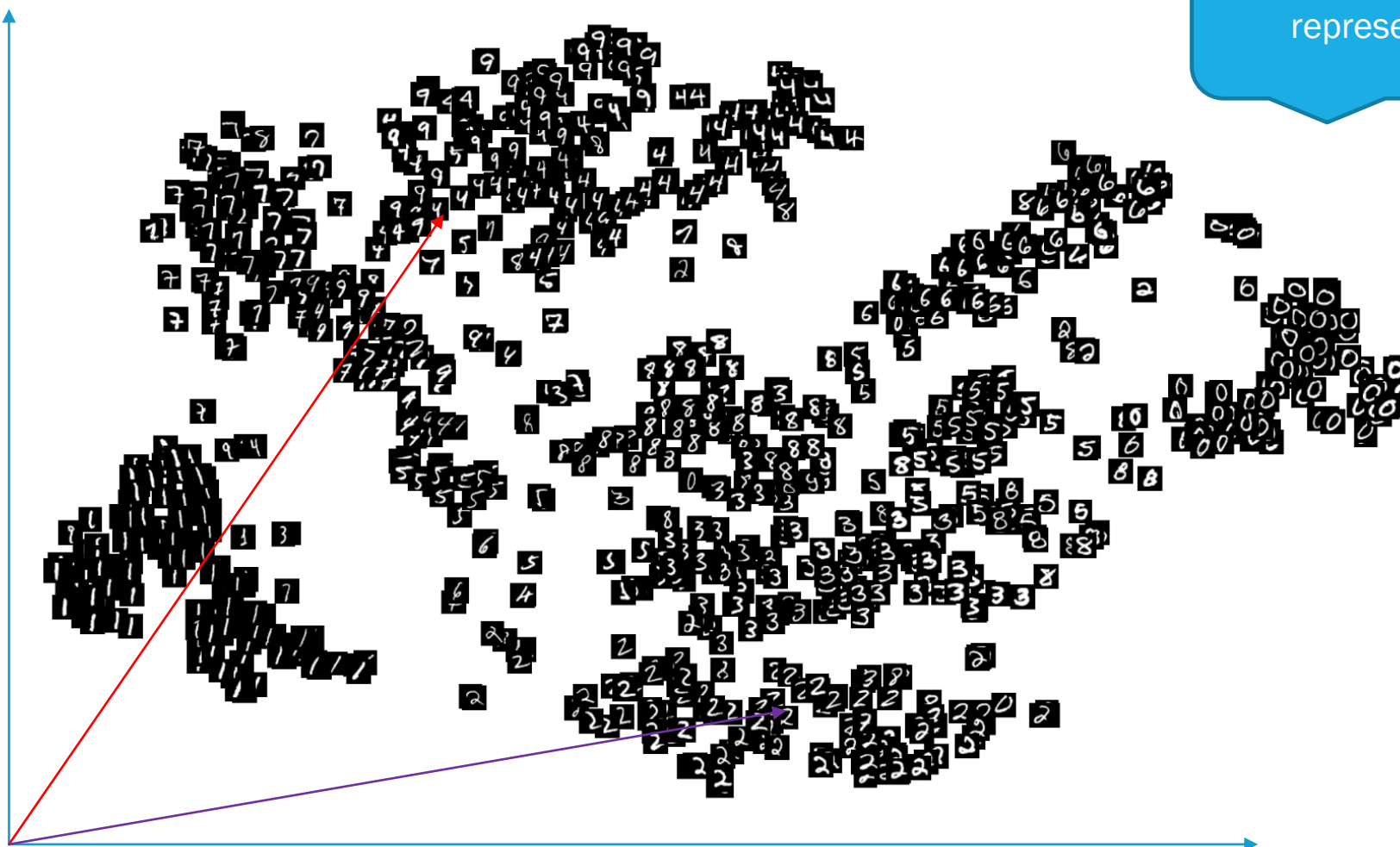
Learned Code is called a Latent representation



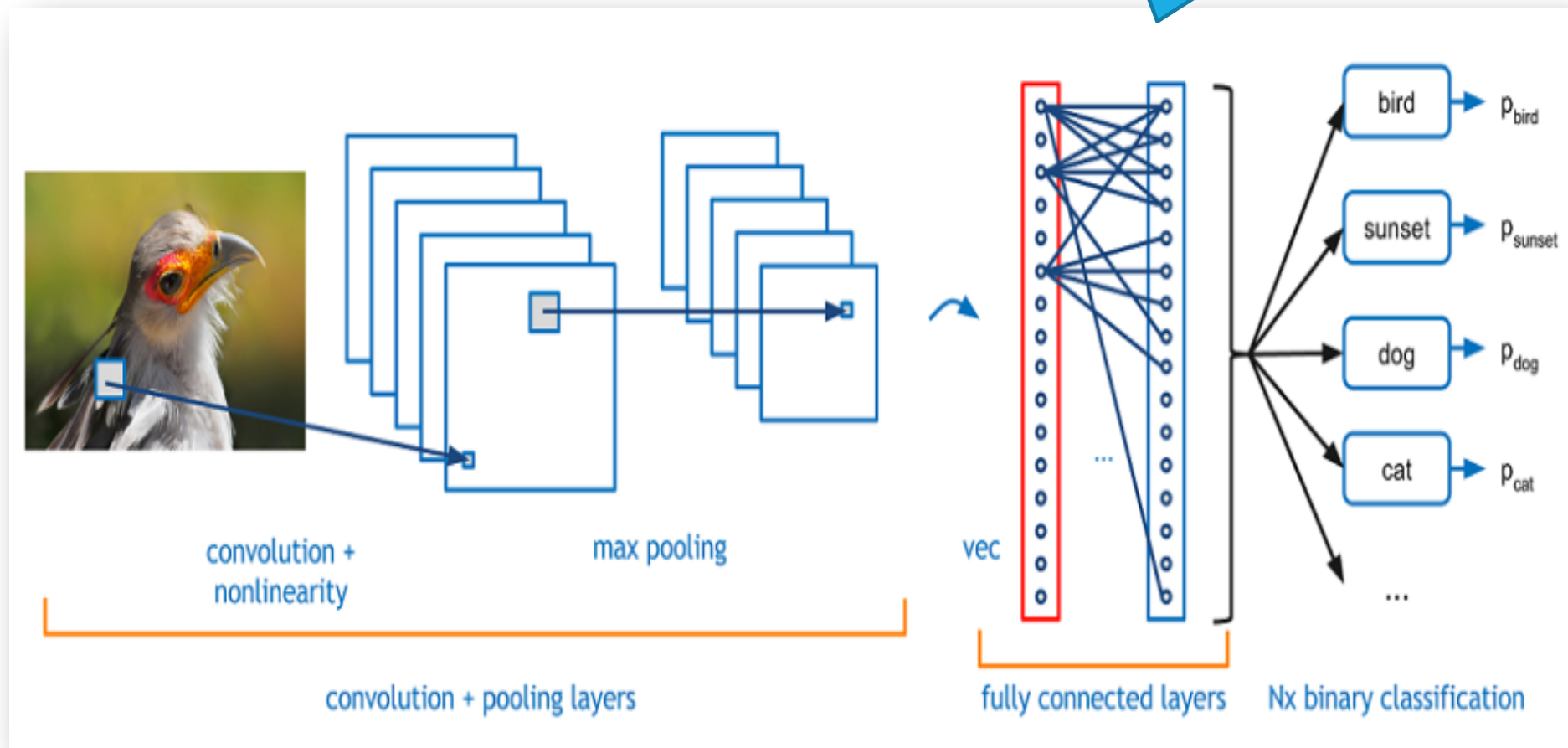
Examples of
reconstructed images
after learning



Latent Space with 2 dimensions

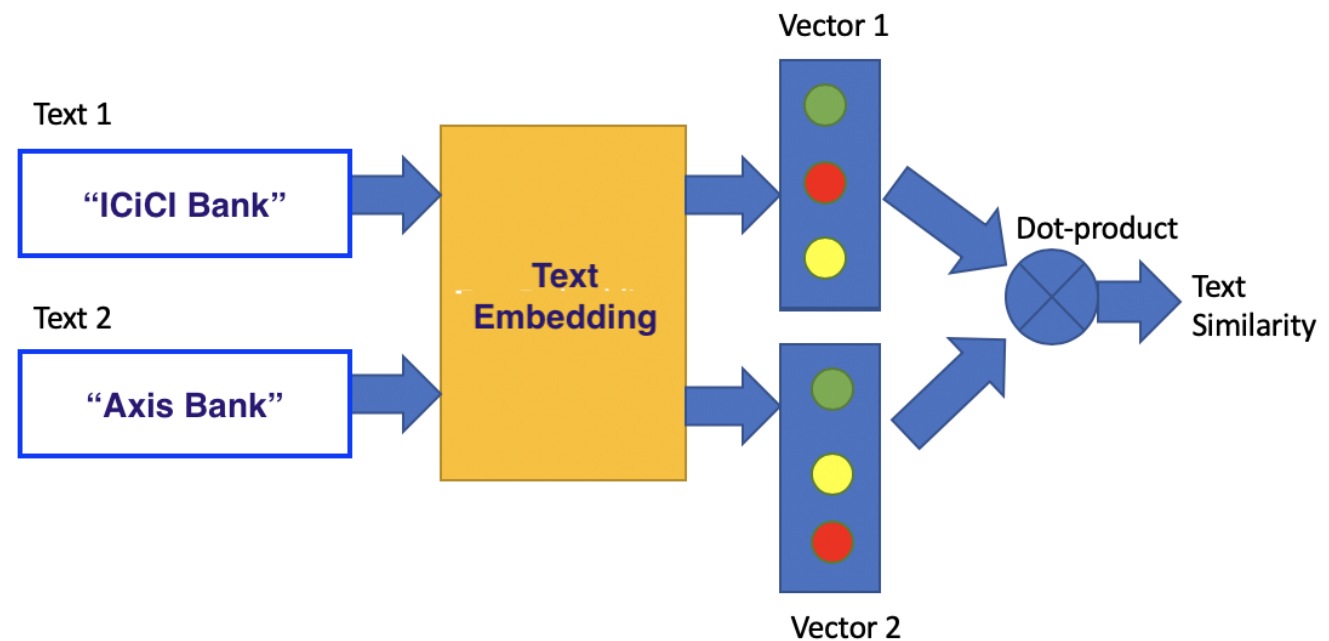


Instead of auto-encoders,
representations can be
learned in CNNs



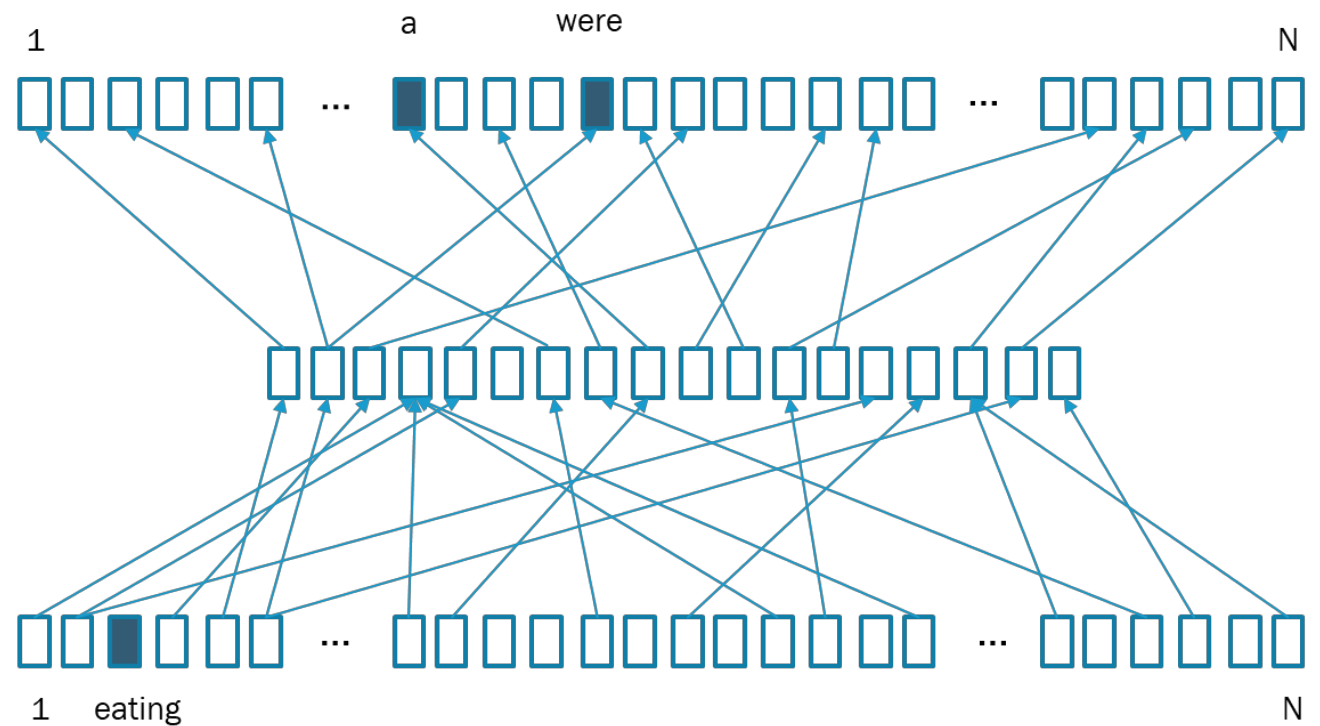
What Are Text Embeddings?

Text embeddings, also known as word embeddings or vector representations, are numerical representations of words or phrases in a high-dimensional space. These representations are designed to capture semantic and syntactic similarities between words, allowing machines to understand the meaning of words based on their context and relationships with other words.

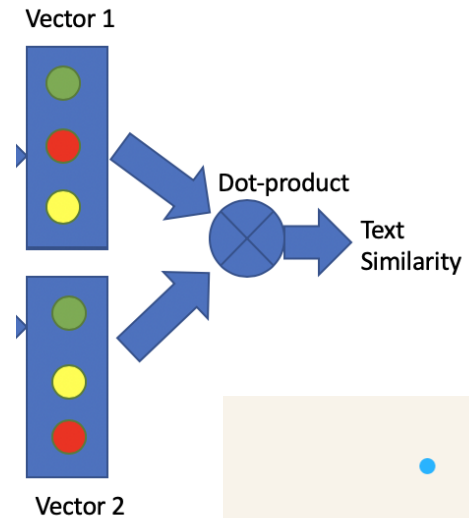


Obtain a very large corpus. Go through all sentences to train a model using a word and its surrounding words.

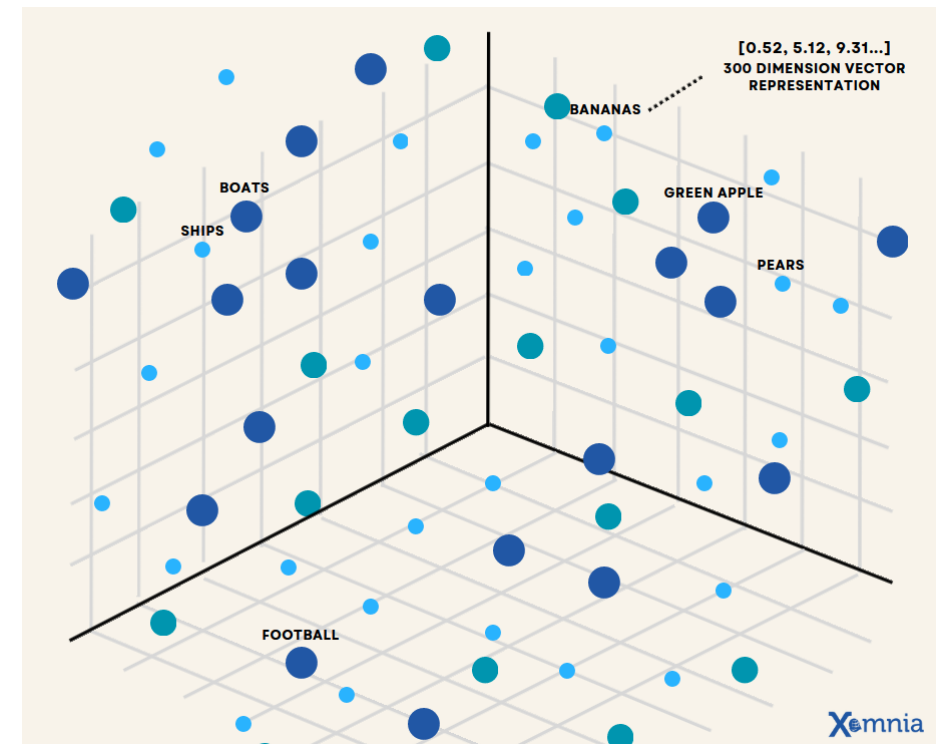
You were eating a red apple this morning.
 You were eating a red apple this morning.
 You were eating a red apple this morning.
 You were eating a red apple this morning.
 You were eating a red apple this morning.
 You were eating a red apple this morning.



word	Embedding (size = 50)
automobile	-0.41195 0.069058 0.26701 0.41424 -0.91901 0.63319 -0.89194 -0.53483 0.19187 -0.038827 1.1475 -0.1396 -0.66392 -0.19639 0.30304 -0.06703 -0.95611 1.6306 0.17545 -1.6013 1.2995 - 1.0079 -1.7455 -0.00058892 -0.021532 -0.97641 -0.93735 0.040884 0.31757 0.55358 1.5822 0.14179 0.37018 0.39469 0.47537 -0.53013 -0.043661 0.42126 0.29403 0.80253 -0.61572 - 0.76155 0.9184 -0.72823 0.59806 -0.16884 -0.59675 0.16543 0.89073 -0.060983
comfort	0.45849 0.91592 -0.27884 -0.67579 1.0385 -0.13239 -0.0029672 0.29708 0.29086 0.14638 0.011615 0.36847 0.69142 -0.61768 -0.10487 0.40521 -0.50348 -0.017701 0.48911 -1.1228 - 0.17567 1.019 -0.80644 -0.16709 0.57987 -0.72542 -0.62351 -0.0096739 1.0891 0.36178 1.9589 1.195 0.038978 -0.1175 0.47923 0.14341 -0.049582 -0.28468 -0.013233 -0.59639 0.36529 0.053435 0.33613 0.3281 0.38041 0.24016 -0.10248 -0.22486 0.24354 0.46712
ralph	-0.89005 -0.28722 -0.29375 -0.20416 1.0173 0.40185 -1.1642 -1.0235 -0.5229 -0.66993 -0.093718 0.275 -0.59325 -0.81409 0.44641 0.091214 0.37466 -0.3537 -0.10802 -0.73682 -0.10244 0.2483 - 0.50697 -0.20149 -0.16329 -0.47134 -0.29646 -0.4719 -0.29636 -0.3282 0.31125 -0.20933 - 0.057354 -1.0292 0.049036 -0.24412 0.017263 0.28363 0.23106 -0.22451 0.023439 0.92904 - 0.69422 -0.90352 0.2107 0.20974 -0.64822 -1.2694 0.24252 1.335
1869	-0.3681 0.32056 -0.7805 -1.0423 -0.27921 0.4636 -0.90183 0.18945 -0.99069 -1.1887 0.1245 0.25086 0.31206 -0.70986 -0.085944 -0.36447 -0.89217 0.0096013 -0.99771 0.96641 1.3968 - 0.7243 0.16906 -0.25744 0.065666 -0.33128 0.36096 -0.62237 0.084945 0.64592 0.66111 - 1.2237 -0.43886 -0.4805 0.060063 -0.45997 0.50485 0.63666 0.40003 0.57787 0.014727 -1.0246 -0.09467 -0.44955 -1.0421 0.34727 -0.35703 -1.2056 0.21645 -0.11689



Learned
embeddings



What Are Item or User Embeddings?

Remember Matrix Factorization?

$$\hat{r}_{ui} = p_u^T q_i$$

User Factors

P	C1	C2	C3
Amit	0.91	1.10	0.51
Meera	1.01	0.12	0.82
Moran	0.75	-0.06	1.13
John	0.35	0.76	0.16

Item Factors

Q						
C1	0.15	1.13	0.91	1.23	0.65	-1.15
C2	0.01	-0.14	1.17	1.52	-0.01	0.01
C3	0.33	0.21	-1.2	1.05	0.89	0.82

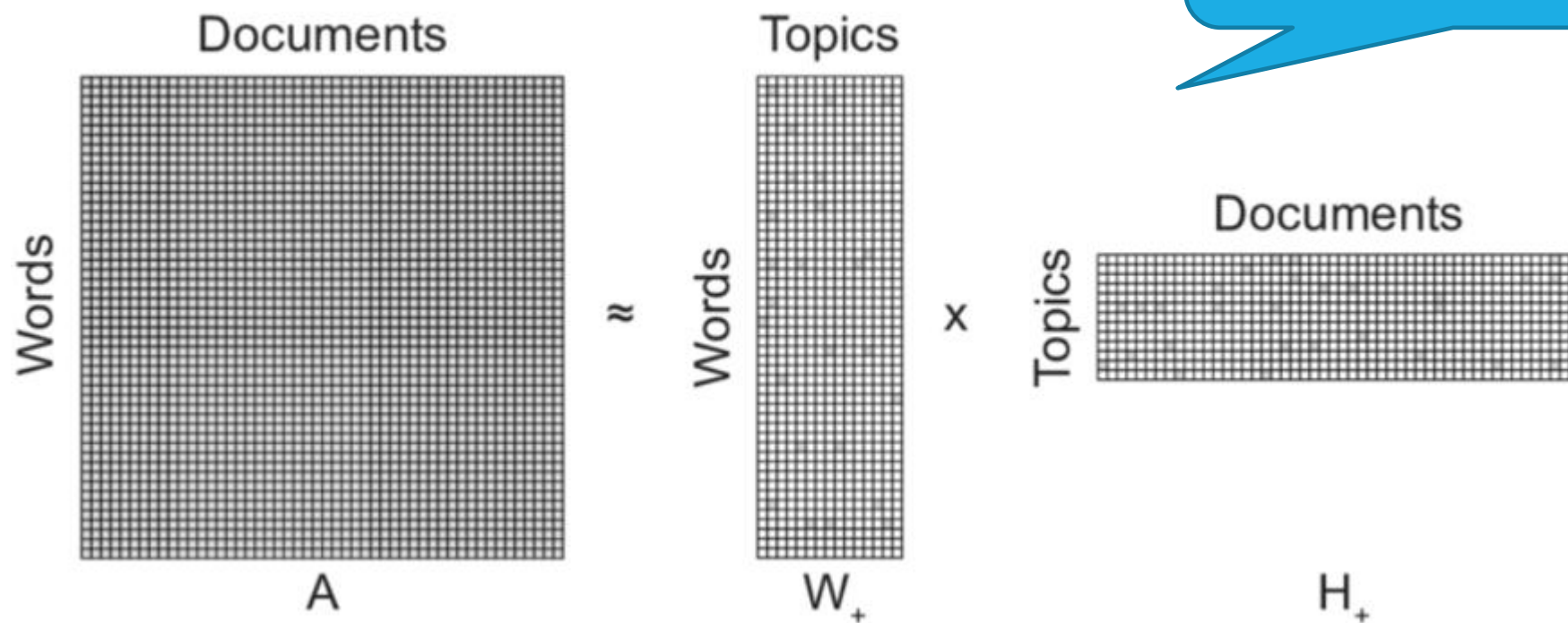
Rating Matrix

R						
Amit	?	?	5	?	4	?
Meera	1	5	?	?	?	4
Moran	1	?	4	5	2	?
John	?	2	3	4	4	?

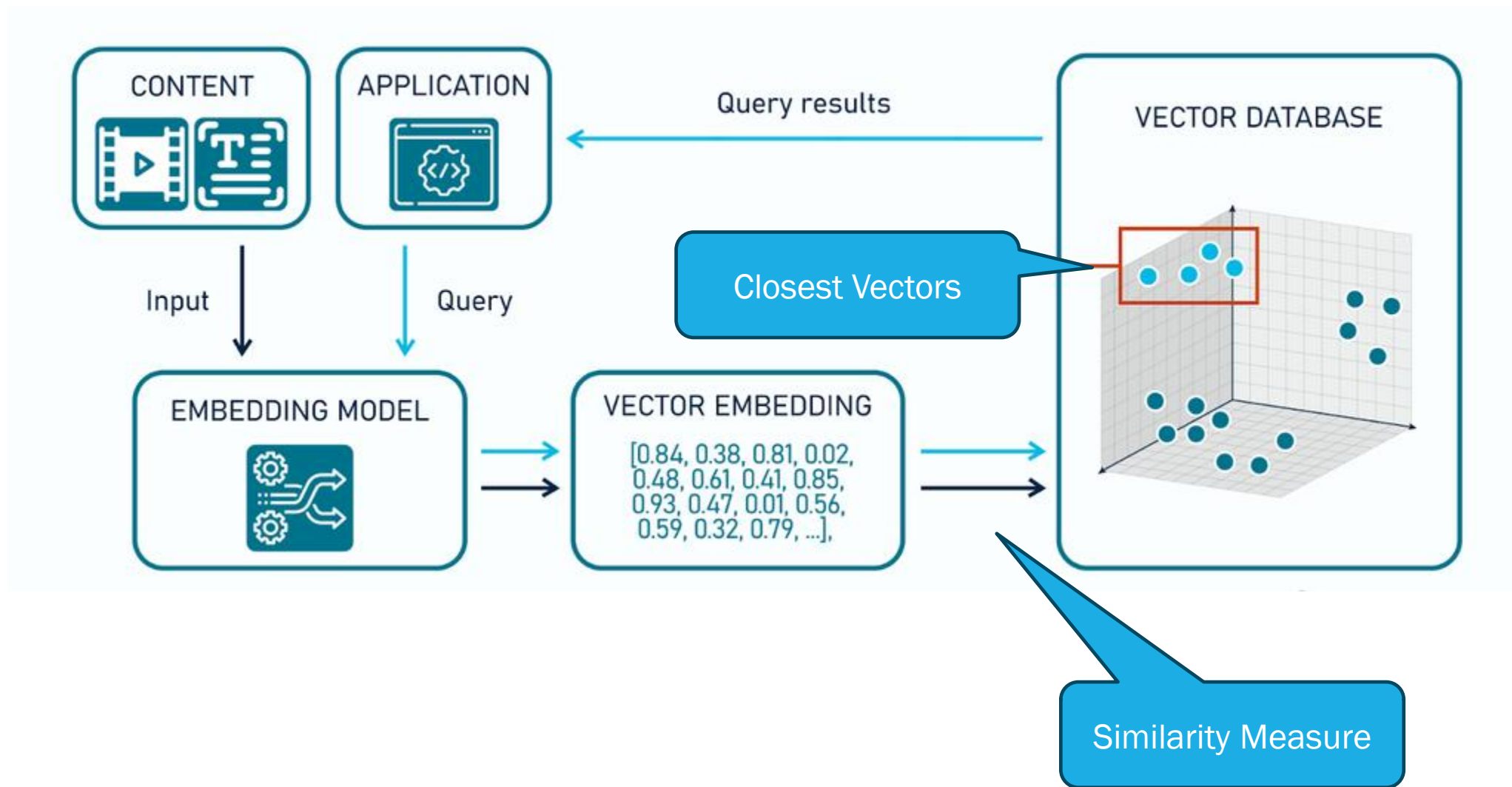
These are latent vectors

What Are Document Embeddings?

$$A \approx W \times H$$



The same idea can be applied to a word-document matrix instead of a user-item matrix



15 Types of Databases and When to Use Them

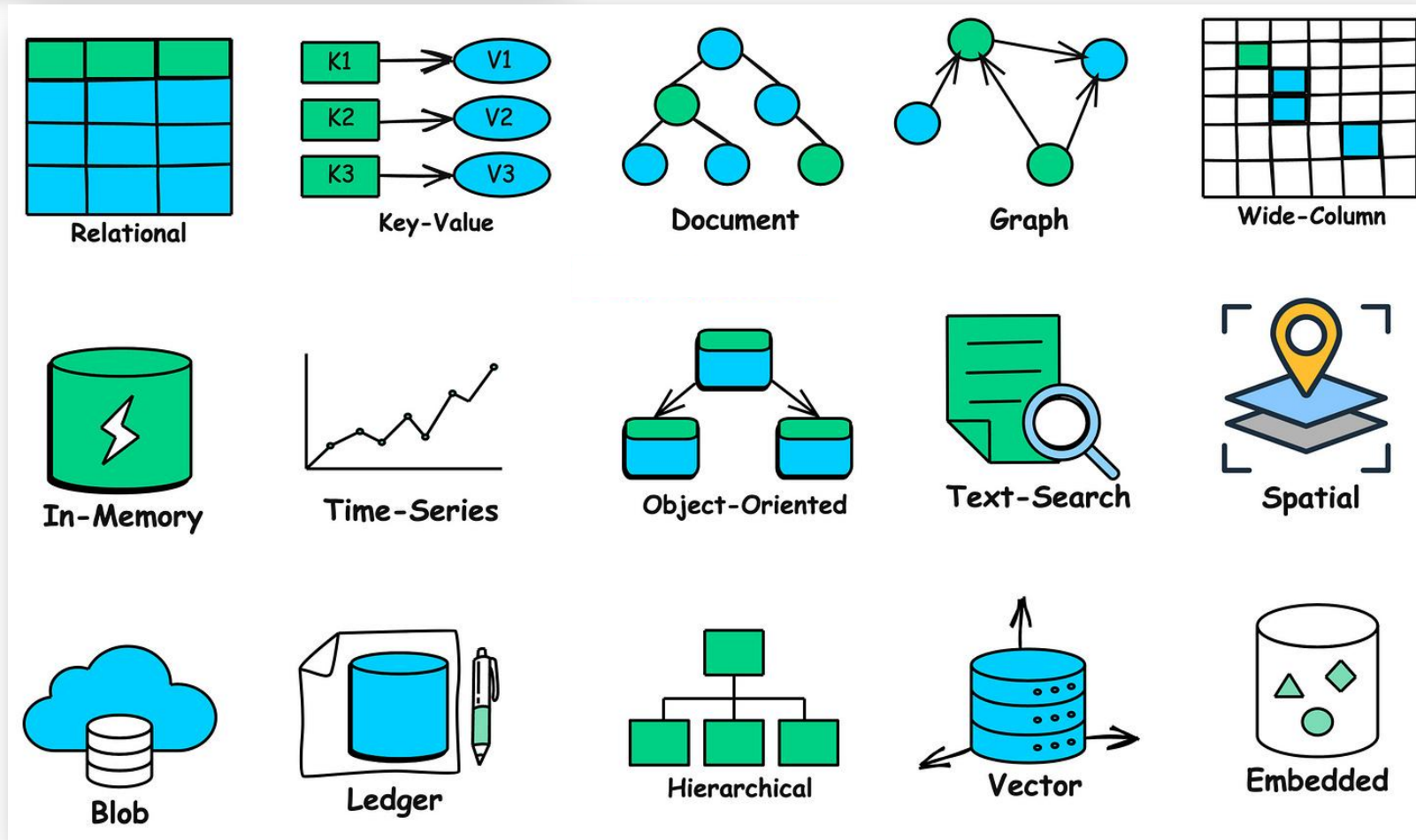
#4 System Design - Types of Databases

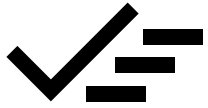


ASHISH PRATAP SINGH
MAR 24, 2024

We went through all of them...

Source





DATA STORAGE

- OLAP/OLTP
- Relational Database
- SQL vs NoSQL
- Expanding to more Data Types and Data Storage